

UNIVERSIDAD DE MÁLAGA
ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA DE TELECOMUNICACIÓN

TRABAJO FIN DE GRADO

SISTEMA DE APRENDIZAJE FEDERADO CON
CLIENTES ANDROID PARA EL ANÁLISIS DE
PARÁMETROS DE COMUNICACIÓN

GRADO EN INGENIERÍA
TELEMÁTICA

ANTONIO JOSÉ MARTÍNEZ HERMOSILLA
MÁLAGA, 2026

Título del Trabajo Fin de Grado

Autor: Antonio José Martínez Hermosilla

Tutor: Gabriel Jesús Luque Polo

Departamento: Lenguajes y Ciencias de la Computación

Titulación: Grado en Ingeniería Telemática

Palabras clave: Aprendizaje Federado, Sistemas Distribuidos, Eficiencia de Red, Cliente y Servidor, API REST, Inteligencia Artificial On-Device.

Resumen

Este Trabajo Fin de Grado tiene como objetivo diseñar e implementar un sistema práctico de aprendizaje federado compuesto por una aplicación Android que actúa como cliente y un servidor central encargado de la agregación de modelos.

El sistema entrena modelos localmente para preservar la privacidad, enviando al servidor solo actualizaciones matemáticas. Además, automatiza experimentos para evaluar el impacto de la red en sistemas distribuidos, midiendo el tráfico, las latencias por el *Straggler Effect* y el rendimiento con datos heterogéneos (Non-IID).

El cliente Android nativo utiliza Kotlin y TensorFlow Lite (LiteRT) para el entrenamiento *on-device*. Por su parte, el servidor en Python (FastAPI) orquesta las rondas y agrega los modelos mediante el algoritmo FedAvg ponderado a través de una API REST.

El resultado es un entorno de experimentación híbrido y funcional que evalúa empíricamente los compromisos entre la convergencia del modelo y el coste de comunicación, validando la eficacia de técnicas de telecomunicación como la compresión y la cuantización para optimizar el consumo de ancho de banda en redes móviles.

English version of the title

Author: Antonio José Martínez Hermosilla

Supervisor: Gabriel Jesús Luque Polo

Department: Languages and Computer Science

Degree: Bachelor's Degree in Telematics Engineering

Keywords: Federated Learning, Distributed Systems, Network Efficiency, Client and Server, REST API, On-Device Artificial Intelligence.

Abstract

This Bachelor's Thesis aims to design and implement a practical federated learning system comprised of an Android application acting as a client and a central server responsible for model aggregation.

The system trains models locally to preserve privacy, sending only mathematical updates to the server. It also automates experiments to evaluate the network's impact on distributed systems, measuring traffic, latency (using the Straggler Effect), and performance with heterogeneous (non-IID) data.

The native Android client uses Kotlin and TensorFlow Lite (LiteRT) for on-device training. Meanwhile, the Python server (FastAPI) orchestrates the training rounds and aggregates the models using the weighted FedAvg algorithm via a REST API.

The result is a functional hybrid experimental environment that empirically evaluates the trade-offs between model convergence and communication cost, validating the effectiveness of telecommunications techniques such as compression and quantization for optimizing bandwidth consumption in mobile networks.

*A mi familia. Y muy especialmente a mis abuelos; sé que hoy estaríais
orgullosos de verme llegar hasta aquí.*

Antonio José Martínez Hermosilla

Agradecimientos

Escribir estas líneas es, sin duda alguna, la parte más bonita y gratificante de todo el proyecto. Cierro una de las etapas más importantes y exigentes de mi vida, y no se me ocurre una forma mejor de hacerlo que recordando a todas las personas que me han dado la fuerza para no rendirme y llegar hasta aquí.

Quiero empezar dándole las gracias a mi familia, y muy especialmente a mi padre y a mi madre. Mamá, papá, sin vosotros esto directamente no habría sido posible. Gracias por el apoyo incondicional, por aguantar mis nervios y por hacer todo lo que estaba en vuestras manos (y muchas veces más) para que yo pudiera centrarme, estudiar y cumplir esta meta. Y junto a vosotros, quiero hacerle una mención especial a mi hermano, Alejandro. Gracias por ser mi confidente, por estar ahí siempre para escucharme (incluso cuando solo me quejaba) y por ser mi ejemplo a seguir. Sinceramente, no podría haber tenido mejor hermano mayor que tú.

A mi novia, Rosa. Gracias por tu paciencia infinita, por aguantar mis picos de estrés, mis ausencias y mis bloqueos. Has sido mi refugio, mi tranquilidad al final del día y la persona que me ha empujado a seguir cuando sentía que no podía más, a pesar de la distancia. Gracias por creer en mí, muchas veces más que yo mismo.

Por supuesto, a mis amigos. Estos años de carrera no habrían sido lo mismo sin vosotros. Gracias por hacer que las horas eternas de biblioteca fueran más llevaderas, por los cafés que nos daban la vida y por todas esas risas que nos servían para desconectar del mundo. De esta etapa me llevo mucho más que un título, me llevo a la mejor gente.

Por último, agradecer a mi tutor Gabriel Luque. Gracias por las enseñanzas transmitidas, por saber guiarme cuando estaba atascado y por exigirme para sacar lo mejor de este proyecto. Tu ayuda y tus consejos han sido fundamentales para darle sentido a todo esto.

Siglas y Acrónimos

API	Application Programming Interface
ARM	Advanced RISC Machine
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSV	Comma-Separated Values
F16	Float16
F32	Float32
FedAvg	Federated Averaging
FL	Federated Learning
HTTP	HyperText Transfer Protocol
IA	Inteligencia Artificial
IID	Independent and Identically Distributed
IoT	Internet of Things
JSON	JavaScript Object Notation
KB	Kilobyte
MB	Megabyte
MLP	Multi-Layer Perceptron
Non-IID	Non-Independent and Identically Distributed
RAM	Random Access Memory
REST	Representational State Transfer
TFG	Trabajo Fin de Grado
TFLite/LiteRT	TensorFlow Lite / Lite Runtime

Índice

Resumen

Abstract

Agradecimientos

Siglas y Acrónimos

I Introducción

Capítulo 1: Introducción y Visión General

1.1.	Contexto y motivación.....	3
1.2.	Objetivos.....	4
1.2.1.	Objetivo general.....	4
1.2.2.	Objetivos específicos.....	4
1.3.	Estado del arte	5
1.4.	Metodología y directrices seguidas	6
1.5.	Estructura del documento.....	6
1.6.	Ámbito de aplicación	7

Capítulo 2: Fundamentos Teóricos

2.1.	Fundamentos de Machine Learning	8
2.2.	Aprendizaje Federado (Federated Learning)	9
2.3.	Distribución de Datos en Sistemas Distribuidos	10
2.4.	Eficiencia y Restricciones en Redes de Comunicación	11
2.5.	Tecnologías y Herramientas Utilizadas.....	12

II Desarrollo del Proyecto

Capítulo 3: Diseño y Arquitectura del Sistema

3.1.	Arquitectura General	16
3.2.	Protocolo de Comunicación y API REST	18

3.3.	Interoperabilidad y Formato de Datos.....	21
3.4.	Control y Sincronización.....	22

Capítulo 4: Implementación del Sistema

4.1.	Servidor Central de Agregación (FastAPI).....	24
4.1.1.	Ciclo de Vida y Recepción de Modelos.....	25
4.1.2.	Control de Rondas Obsoletas y Robustez Temporal	26
4.1.3.	Algoritmo de Agregación Ponderada (FedAvg).....	27
4.1.4.	Subsistema de Telemetría y Exportación de Logs.....	28
4.2.	Cliente Android (Kotlin + TFLite/LiteRT).....	29
4.2.1.	Arquitectura de la App y Módulo	31
4.2.2.	Solución a Retos Técnicos de Ingeniería.....	32
4.3.	Herramientas de Automatización	34
4.3.1.	Orquestador de Experimentos (<i>ejecutar_experimentos.py</i>)	35
4.3.2.	Módulo de Post-procesamiento y Generación de Gráficas (<i>generar_graficas.py</i>)	36

III Experimentación

Capítulo 5: Plan de Experimentación y Análisis de Resultados

5.1.	Metodología de Evaluación.....	42
5.1.1.	Parámetros Configurables del Sistema.....	43
5.1.2.	Métricas Telemáticas y de Rendimiento Registradas	43
5.1.3.	Validez Estadística mediante Control de Semillas	44
5.2.	Diseño del Bloque 1: Hardware Real y Cuellos de Botella Físicos 45	
5.2.1.	Experimento 1.1: Validación del Entrenamiento Real <i>On-Device</i> en Cliente Android Solitario.....	46
5.2.2.	Experimento 1.2: Aislamiento y Cuantificación del Efecto Rezagado (<i>Straggler Effect</i>).....	47

5.3. Diseño del Bloque 2: Simulaciones a Escala y Eficiencia de Red47

5.3.1. Experimento 2.1: Escenarios de Distribución de Datos (IID vs. Non-IID Patológico).....	49
5.3.2. Experimento 2.2: Compromiso entre Cómputo Local y Comunicación (<i>Trade-off</i>).....	49
5.3.3. Experimento 2.3: Evaluación Comparativa de Técnicas de Optimización Telemática	50
5.3.4. Experimento 2.4: Escalabilidad del Tráfico de Red frente al Aumento de Nodos	50

5.4. Diseño del Bloque 3: Flotas Mixtas y Agregación Desbalanceada 51

5.4.1. Experimento 3.1: Flota Híbrida Equilibrada (Proporción 1:1).....	52
5.4.2. Experimento 3.2: Flotas Híbridas Desbalanceadas (Proporciones 1:10 y 1:50)	53

Capítulo 6: Análisis de los Resultados

6.1. Análisis del Bloque 1: Hardware Real y Cuellos de Botella Físicos 55

6.1.1. Evaluación de la Convergencia Real On-Device (Exp. 1.1).....	55
6.1.2. Demostración Empírica y Cuantificación del Efecto <i>Straggler</i> (Exp. 1.2)	57

6.2. Análisis del Bloque 2: Simulaciones a Escala y Eficiencia de Red 59

6.2.1. Divergencia de Pesos en Entornos IID vs. Non-IID (Exp. 2.1)....	60
6.2.2. Rentabilidad Temporal y Ahorro de Ancho de Banda (Exp. 2.2).	65
6.2.3. Comparativa de Compresión y Cuantización (Exp. 2.3)	68
6.2.4. Escalabilidad del Tráfico y Degradación Asintótica (Exp. 2.4)	72

6.3. Análisis del Bloque 3: Flotas Mixtas y Agregación Desbalanceada 76

6.3.1.	Convergencia en Escenarios Equilibrados 1:1 (Exp. 3.1)	76
6.3.2.	Simbiosis y Asimilación del FedAvg Ponderado 1:10 y 1:50 (Exp. 3.2)	78

IV Conclusiones y Líneas Futuras

Capítulo 7: Conclusiones y Trabajo Futuro

7.1.	Conclusiones del Proyecto	86
7.2.	Cumplimiento de Objetivos.....	87
7.3.	Líneas Futuras.....	88
7.3.1.	Estrategias Asíncronas de Agregación (Mitigación del <i>Straggler Effect</i>)	88
7.3.2.	Sustitución de HTTP REST por gRPC y Protocol Buffers (ProtoBuf)	89
7.3.3.	Participación Parcial Dinámica y Tolerancia al Dropout.....	89
7.3.4.	Aprendizaje Federado Personalizado (Personalized Federated Learning, PFL)	90

V Apéndices y Bibliografía

Apéndice A. Configuración y Despliegue del entorno

Bibliografía

Índice de Figuras

Figura 3.1.1: Esquema General del Sistema.....	16
Figura 3.2.1: Ciclo de Vida de una Ronda.....	19
Figura 3.4.1: Ciclo Early Stopping y Rondas_Ojetivos	23
Figura 4.1.1: Esquema Servidor ASGI con Uvicorn	25
Figura 4.2.1: Arquitectura app movil android LiteRT	30
Figura 4.3.1: Flujo de Automatización y Análisis	34
Figura 4.3.2: Flujo de automatización de experimentos	35
Figura 5.1.1: Parámetros de Evaluación para pruebas de Experimentación....	42
Figura 5.2.1: Experimentos Bloque 1	46
Figura 5.3.1: Experimentos Bloque 2	48
Figura 5.4.1: Experimentos Bloque 3	51
Figura 6.1.1: Captura Terminal Ronda 1 Experimento 1.1	55
Figura 6.1.2: Captura Terminal Ronda 50 Experimento 1.1	55
Figura 6.1.3: Gráfica Accuracy Experimento 1.1	56
Figura 6.1.4: Gráfica Tiempo Ejecución por Ronda Experimento 1.1.....	56
Figura 6.1.5: Accuracy Experimento 1.2	58
Figura 6.1.6: Loss Experimento 1.2.....	58
Figura 6.1.7: Tiempo Ejecución por Ronda Experimento 1.2	58
Figura 6.1.8: Tráfico Acumulado Experimento 1.2	58
Figura 6.2.1: Accuracy IID Experimento 2.1	61
Figura 6.2.2: Loss IID Experimento 2.1	61
Figura 6.2.3: Tiempo Ejecución por Ronda IID Experimento 2.1.....	61
Figura 6.2.4: Tráfico Acumulado IID Experimento 2.1	61
Figura 6.2.5: Accuracy Non-IID Experimento 2.1	63
Figura 6.2.6: Loss Non-IID Experimento 2.1	63
Figura 6.2.7: Tiempo Ejecución por Ronda Non-IID Experimento 2.1.....	64
Figura 6.2.8: Tráfico Acumulado Non-IID Experimento 2.1	64
Figura 6.2.9: Accuracy Experimento 2.2	66
Figura 6.2.10: Loss Experimento 2.2.....	66
Figura 6.2.11: Tiempo Ejecución por Ronda Experimento 2.2	66
Figura 6.2.12: Tráfico Acumulado Experimento 2.2	66

Figura 6.2.13: Accuracy Experimento casos 2.3	68
Figura 6.2.14: Loss Experimento casos 2.3	69
Figura 6.2.15: Tiempo Ejecución por Ronda Experimento casos 2.3.....	69
Figura 6.2.16: Tráfico Acumulado Experimento casos 2.3	70
Figura 6.2.17: Gráfica Comparativa Cuantización y Compresión Experimento 2.3	71
Figura 6.2.18: Accuracy Experimentos 2.4.....	72
Figura 6.2.19: Loss Experimentos 2.4.....	73
Figura 6.2.20: Tiempo Ejecución por Ronda Experimentos 2.4	73
Figura 6.2.21: Tráfico Acumulado Experimentos 2.4.....	74
Figura 6.2.22: Gráfica Tráfico y Precisión con Escalabilidad del Clúster Experimento 2.4	75
Figura 6.3.1: Accuracy Experimento 3.1	77
Figura 6.3.2: Loss Experimento 3.1.....	77
Figura 6.3.3: Tiempo Ejecución por Ronda Experimento 3.1	77
Figura 6.3.4: Tráfico Acumulado Experimento 3.1	77
Figura 6.3.5: Accuracy 1:10 Experimento 3.2.....	78
Figura 6.3.6: Loss 1:10 Experimento 3.2.....	79
Figura 6.3.7: Tiempo Ejecución por Ronda 1:10 Experimento 3.2	79
Figura 6.3.8: Tráfico Acumulado 1:10 Experimento 3.2	79
Figura 6.3.9: Accuracy 1:50 Experimento 3.2.....	80
Figura 6.3.10: Loss 1:50 Experimento 3.2.....	80
Figura 6.3.11: Tiempo Ejecución por Ronda 1:50 Experimento 3.2	81
Figura 6.3.12: Tráfico Acumulado 1:50 Experimento 3.2	81
Figura 6.3.13: Gráfica Proporción Asignación Muestras Experimentos 3.1 y 3.2	82

Parte I

Introducción

Capítulo 1

Introducción y visión general

Contenido

1.1.	Contexto y motivación.....	3
1.2.	Objetivos.....	4
1.2.1.	Objetivo general.....	4
1.2.2.	Objetivos específicos	4
1.3.	Estado del arte	5
1.4.	Metodología y directrices seguidas	6
1.5.	Estructura del documento.....	6
1.6.	Ámbito de aplicación	7

1.1. Contexto y motivación

El despliegue de modelos de aprendizaje automático (del inglés *Machine Learning*, ML) experimenta un crecimiento constante en servicios reales. Tradicionalmente, su entrenamiento requiere la centralización masiva de datos en servidores, lo que exige un elevado ancho de banda, un alto coste computacional y, fundamentalmente, suscita severos riesgos de privacidad al tratar información sensible de los usuarios [1].

Simultáneamente, la evolución del hardware ha dotado a los dispositivos móviles de una notable capacidad de procesamiento en el borde de la red (*Edge Computing*) [2]. En este contexto emerge el Aprendizaje Federado (FL, del inglés *Federated Learning*) [3], un paradigma descentralizado donde múltiples dispositivos colaboran en el entrenamiento de un modelo global. En un sistema federado, los nodos ejecutan la optimización localmente con sus propios datos y

devuelven al servidor únicamente las actualizaciones matemáticas (pesos y sesgos), preservando la privacidad por diseño.

Sin embargo, la implementación del Aprendizaje Federado impone importantes desafíos en el ámbito de la Ingeniería Telemática. La eficiencia de la red queda condicionada por el volumen de información intercambiada y las latencias de comunicación. Concretamente, la heterogeneidad de los dispositivos introduce cuellos de botella como el efecto rezagado (*Straggler Effect*), donde los terminales con menor capacidad de cómputo penalizan el tiempo de sincronización global del sistema. Por consiguiente, resulta imprescindible evaluar los parámetros de comunicación para lograr un equilibrio viable entre la convergencia del modelo y el consumo de recursos de red.

1.2. Objetivos

1.2.1. Objetivo general

El objetivo principal de este Trabajo Fin de Grado (TFG) radica en el diseño, implementación y evaluación telemática de un sistema práctico e híbrido de Aprendizaje Federado, constituido por una aplicación cliente para dispositivos móviles Android y un servidor central de agregación. A partir de la construcción de esta arquitectura, se pretende analizar el impacto empírico de distintos parámetros de comunicación y configuraciones de red sobre el coste de transmisión y la calidad de las predicciones del modelo resultante, orientando el estudio hacia la optimización y la eficiencia de los sistemas distribuidos.

1.2.2. Objetivos específicos

Para alcanzar la meta principal expuesta, el proyecto se desglosa en la consecución de los siguientes objetivos específicos:

- Diseñar la arquitectura lógica de un sistema cliente-servidor distribuido orientado al entrenamiento seguro de modelos neuronales.

- Implementar un servidor central mediante una API REST (del inglés *Representational State Transfer*) que gestione las rondas de entrenamiento, agregue los modelos locales mediante el algoritmo FedAvg ponderado y distribuya la versión global.
- Desarrollar una aplicación móvil nativa en Android capaz de ejecutar el entrenamiento local en el procesador del dispositivo (*on-device*) y de transmitir sus parámetros actualizados.
- Evaluar el impacto de la reducción de la carga telemática mediante técnicas de cuantización numérica y compresión sobre los paquetes de red.
- Simular escenarios de red heterogéneos, analizando el efecto de los clientes rezagados (*Straggler Effect*) y el comportamiento del sistema ante distribuciones de datos patológicas o sesgadas (*Non-IID*).
- Automatizar baterías de experimentos empíricos reproducibles para contrastar el rendimiento del clúster.
- Analizar los resultados mediante métricas telemáticas estandarizadas: tráfico de red generado, tiempo de latencia, coste computacional y precisión final del modelo de clasificación.

1.3. Estado del arte

El concepto de Aprendizaje Federado fue introducido inicialmente como una solución para entrenar modelos predictivos en teclados de dispositivos móviles sin necesidad de extraer el texto tecleado por los usuarios [3]. Desde entonces, este paradigma ha evolucionado hacia un campo de investigación central en la Inteligencia Artificial en el Borde (*Edge AI*), impulsado por el endurecimiento de las normativas internacionales de protección de datos.

Actualmente, el estado de la técnica se divide en dos grandes vertientes. Por un lado, la optimización puramente matemática, enfocada en mejorar la convergencia de los modelos cuando los datos locales presentan distribuciones heterogéneas y no independientes (*Non-IID*) [4]. Por otro lado, la vertiente de la

Ingeniería de Telecomunicación, centrada en mitigar los cuellos de botella de red. Las investigaciones recientes proponen técnicas de reducción de tráfico, como la cuantización de pesos sinápticos y la compresión de gradientes, así como estrategias de agregación asíncrona para paliar las latencias inducidas por terminales móviles con recursos limitados.

1.4. Metodología y directrices seguidas

Para la consecución de los objetivos propuestos, se ha seguido una metodología de desarrollo iterativa e incremental, fundamentada en principios de ingeniería telemática. El proyecto se ha estructurado en fases de complejidad ascendente:

1. Asentamiento de bases teóricas mediante simulaciones locales.
2. Diseño y despliegue de la arquitectura de red mediante una API REST.
3. Implementación de agentes autónomos y adaptación del entorno de ejecución *on-device* en Android.
4. Automatización y validación empírica.

La experimentación se ha conducido bajo un rigor estadístico estricto. Para garantizar la reproducibilidad de los escenarios y aislar las variables de red de las fluctuaciones matemáticas, se ha implementado un control absoluto mediante semillas estadísticas (*seeds*) en todos los nodos distribuidos. La evaluación del sistema no se limita a la tasa de acierto del modelo de aprendizaje, sino que prioriza la monitorización de métricas de red y rendimiento hardware, tales como el tráfico acumulado, los tiempos de procesamiento local y las latencias de sincronización.

1.5. Estructura del documento

La presente memoria se encuentra organizada en las siguientes partes y capítulos:

- **Parte I: Introducción.** Comprende el actual Capítulo 1, donde se expone la motivación, objetivos y metodología, y el Capítulo 2, que detalla los

fundamentos teóricos del Aprendizaje Federado y el estado del arte de las tecnologías implicadas.

- **Parte II: Desarrollo del proyecto.** Abarca el Capítulo 3, dedicado al diseño lógico, la arquitectura de red y el protocolo de comunicación, y el Capítulo 4, donde se describe la implementación técnica del servidor central y del cliente Android, detallando las soluciones adoptadas ante retos de interoperabilidad y compilación.
- **Parte III: Experimentación.** Incluye el Capítulo 5, que define el plan de pruebas, los escenarios diseñados y la metodología de evaluación, y el Capítulo 6, donde se analizan empíricamente los resultados obtenidos en términos de convergencia y eficiencia telemática.
- **Parte IV: Conclusiones y líneas futuras.** Formada por el Capítulo 7, que sintetiza los hallazgos del trabajo, verifica el cumplimiento de objetivos y propone vías de continuación.
- **Parte V: Apéndices y Bibliografía.** Contiene información complementaria sobre el despliegue del entorno y las referencias utilizadas a lo largo del documento.

1.6. Ámbito de aplicación

La arquitectura desarrollada y los resultados obtenidos en este trabajo resultan de aplicación directa en ecosistemas de Internet de las Cosas (IoT) y redes móviles de nueva generación (4G/5G/6G), donde el ancho de banda constituye un recurso crítico. El sistema es extrapolable a sectores que exigen una estricta confidencialidad de la información, tales como el ámbito sanitario (análisis de datos biométricos en *wearables*), la domótica inteligente y las finanzas descentralizadas [5]. Asimismo, proporciona un entorno base funcional para que futuros investigadores evalúen nuevos algoritmos de compresión o protocolos de comunicación en escenarios de computación colaborativa en el borde.

Capítulo 2

Fundamentos Teóricos

Contenido

2.1.	Fundamentos de Machine Learning	8
2.2.	Aprendizaje Federado (Federated Learning)	9
2.3.	Distribución de Datos en Sistemas Distribuidos	10
2.4.	Eficiencia y Restricciones en Redes de Comunicación	11
2.5.	Tecnologías y Herramientas Utilizadas.....	12

2.1. Fundamentos de Machine Learning

El aprendizaje automático (del inglés *Machine Learning*, ML) es una disciplina de la IA que permite a los sistemas computacionales identificar patrones complejos a partir de conjuntos de datos sin haber sido explícitamente programados para ello [6]. Dentro de sus paradigmas, el aprendizaje supervisado constituye uno de los enfoques más extendidos. En este modelo, el sistema se alimenta mediante un conjunto de datos (dataset) compuesto por pares de información: una entrada cruda de datos (vector de variables X) y su correspondiente etiqueta o respuesta correcta (vector de variables Y).

La arquitectura clásica para abordar problemas de clasificación es la red neuronal artificial, siendo el Perceptrón Multicapa (MLP, del inglés *Multi-Layer Perceptron*) su estructura base. Un MLP se organiza en capas consecutivas de neuronas (capa de entrada, capas ocultas y capa de salida). Matemáticamente, la información transita entre capas a través de conexiones sinápticas, donde cada conexión posee un peso numérico (w_i) y cada neurona incorpora un sesgo interno (b_i). La activación de una neurona se determina mediante una

combinación lineal seguida de una función de activación no lineal, expresada genéricamente como:

$$z = \sum_{i=1}^n (w_i \cdot x_i) + b_i \quad (2.1)$$

El proceso de "aprender" consiste en ajustar iterativamente estos miles de parámetros decimales (w y b), inicialmente aleatorios, hasta que las predicciones de la red neuronal coincidan con las etiquetas reales del *dataset*. Para cuantificar el margen de error, se emplea una función de pérdida (*Loss function*), cuyo valor se busca minimizar a través de algoritmos de optimización (como el descenso de gradiente estocástico o Adam) [7]. Por su parte, la precisión de acierto (*Accuracy*) proporciona una métrica comprensible del porcentaje de clasificaciones correctas efectuadas por el modelo.

Durante el entrenamiento local, el conjunto de datos no se procesa en su totalidad de manera simultánea, sino que se divide en subconjuntos de menor tamaño denominados *batches*. Se define como época (*epoch*) al ciclo completo en el que la red neuronal ha procesado la totalidad del conjunto de datos disponible una única vez.

2.2. Aprendizaje Federado (Federated Learning)

El Aprendizaje Federado (FL, del inglés *Federated Learning*) es una técnica de inteligencia artificial distribuida que permite entrenar un modelo global en múltiples dispositivos periféricos sin necesidad de centralizar sus datos locales. Esta topología mitiga de forma nativa los riesgos de privacidad, puesto que las muestras de información (imágenes, textos o sensores) nunca abandonan el dispositivo del usuario [3].

El ciclo de vida estándar de una arquitectura federada se articula en rondas de comunicación síncronas. En cada iteración, un servidor central distribuye el estado actual del modelo global a un conjunto de dispositivos cliente. Estos clientes ejecutan un entrenamiento local durante un número definido de épocas,

utilizando exclusivamente sus datos privados para computar un nuevo conjunto de pesos sinápticos. Finalmente, los clientes transmiten estas actualizaciones numéricas de vuelta al servidor, donde se combinan para generar un modelo global refinado.

El algoritmo de agregación fundamental en este paradigma es el Promediado Federado, conocido como FedAvg [3]. En escenarios reales, la cantidad de datos disponibles varía drásticamente entre dispositivos. Por consiguiente, FedAvg no aplica una media aritmética simple, sino una media ponderada en función del volumen de muestras aportado por cada nodo, descrita matemáticamente por la siguiente ecuación:

$$w_{t+1} = \sum_{k=1}^K \frac{n_k}{n} \times w_t^k \quad (2.2)$$

donde w_{t+1} representa los pesos del modelo global en la ronda siguiente, K es el número de clientes participantes, n_k designa la cantidad de muestras locales del cliente k , y n denota el total de muestras procesadas en la ronda.

2.3. Distribución de Datos en Sistemas Distribuidos

A diferencia de los entornos centralizados en laboratorios, las redes descentralizadas se caracterizan por una extrema heterogeneidad estadística [8]. El rendimiento y la capacidad de convergencia de un sistema federado se ven severamente afectados por la forma en que los datos se distribuyen entre los clientes.

- **Entornos IID (Independientes e Idénticamente Distribuidos):** Se define como un escenario ideal donde cada nodo posee un subconjunto de datos estadísticamente representativo y uniforme respecto a la población global (por ejemplo, conteniendo muestras balanceadas de todas las clases posibles). En estas condiciones, los gradientes locales calculados por los clientes apuntan hacia un mínimo global coincidente, garantizando una convergencia estable y de baja varianza tras la agregación en el servidor.

- **Entornos Non-IID (No Independientes e Idénticamente Distribuidos):** Constituye el escenario realista por excelencia, donde los datos locales presentan fuertes sesgos inherentes al comportamiento de cada usuario. En casos de partición patológica (por ejemplo, cuando un dispositivo solo interactúa con un par de clases específicas), la actualización de pesos de cada nodo tira del modelo global en direcciones divergentes. Este fenómeno, conocido como divergencia de pesos (*Weight Divergence*), induce inestabilidad en el algoritmo FedAvg, provocando oscilaciones en la curva de aprendizaje y exigiendo un número significativamente mayor de rondas de telecomunicación para alcanzar la convergencia [9].

2.4. Eficiencia y Restricciones en Redes de Comunicación

Desde la perspectiva de la Ingeniería Telemática, la transmisión iterativa de matrices de pesos neuronales entre múltiples nodos y un servidor central suscita importantes desafíos de escalabilidad. A diferencia del cómputo puro, la infraestructura de telecomunicaciones impone límites físicos dictados por el ancho de banda, la latencia y la intermitencia del canal.

Para optimizar el uso de los recursos radioeléctricos y reducir la huella de transmisión, se emplean principalmente dos técnicas [10]:

- **Compresión de datos:** Aplicación de algoritmos sin pérdidas (*lossless*), como el formato ZIP, para reducir el tamaño del conjunto de bytes encapsulados (*payload*) justo antes de su transmisión a través de los *sockets* de red.
- **Cuantización de pesos:** Técnica que recorta la precisión aritmética de los tensores matemáticos. Por defecto, las redes neuronales operan en formato Float32 (punto flotante de precisión simple, ocupando 4 bytes por parámetro). Cuantizar el modelo a Float16 (2 bytes por parámetro) recorta a la mitad el tráfico total exigido en la red, asumiendo teóricamente un impacto marginal en la precisión de clasificación del modelo.

Adicionalmente, el despliegue de sistemas síncronos se ve penalizado por el efecto rezagado (*Straggler Effect*). En una ronda estándar, el servidor central no puede efectuar la agregación FedAvg hasta haber recibido la respuesta del último cliente esperado. Dado que las flotas móviles son heterogéneas, la disparidad en las capacidades de la CPU, la RAM y la conectividad de los terminales provoca que el tiempo total de la iteración quede acotado superiormente por el nodo más lento, dilapidando una fracción considerable del ciclo en tiempos de espera inactivos.

2.5. Tecnologías y Herramientas Utilizadas

Para el diseño, simulación y despliegue físico del presente Trabajo de Fin de Grado, se ha configurado un entorno de desarrollo estructurado en dos vertientes tecnológicas complementarias:

- **Infraestructura del Servidor Central:** Desarrollada íntegramente en Python, se ha empleado el marco de trabajo FastAPI [11] para el diseño de la API REST, operando sobre el servidor ASGI Uvicorn para garantizar un alto rendimiento en la gestión de peticiones asíncronas. La evaluación del modelo global y el procesado matricial se fundamentan en las bibliotecas de cálculo científico TensorFlow [1], Keras y NumPy [12].
- **Infraestructura de los Clientes Móviles:** El desarrollo nativo para el sistema operativo Android se ha llevado a cabo mediante el lenguaje Kotlin y el entorno Android Studio. La orquestación de red y la construcción de peticiones HTTP multiparte se han gestionado mediante la librería OkHttp. De manera crítica, la ejecución de la retropropagación en el propio dispositivo (*on-device training*) se ha posibilitado gracias al motor TensorFlow Lite (LiteRT) [13], habilitando experimentalmente el soporte para operaciones extendidas de TensorFlow (SELECT_TF_OPS) orientadas al entrenamiento de variables mutables en arquitecturas ARM.

Parte II

Desarrollo del Proyecto

Capítulo 3

Diseño y Arquitectura del Sistema

Contenido

3.1.	Arquitectura General	16
3.2.	Protocolo de Comunicación y API REST	18
3.3.	Interoperabilidad y Formato de Datos.....	21
3.4.	Control y Sincronización.....	22

3.1. Arquitectura General

El sistema se ha diseñado bajo una topología distribuida de tipo cliente-servidor, orientada a soportar un entorno de ejecución híbrido. La arquitectura equilibra la precisión de las pruebas reales en dispositivos físicos con la capacidad de simulación a escala masiva en entorno de laboratorio [3].

El sistema se descompone en tres entidades funcionales principales:

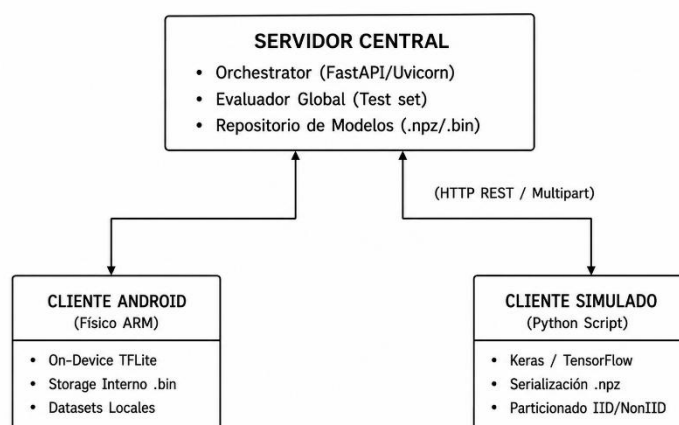


Figura 3.1.1: Esquema General del Sistema

1. **Servidor Central de Agregación:** Actúa como el nodo maestro e inmutable de la red. Es responsable de coordinar el avance de las rondas globales de entrenamiento, almacenar y distribuir el estado del modelo global actual mediante una API REST, recibir y validar las actualizaciones de los clientes, agregar las actualizaciones mediante FedAvg, ponderándolas según el número de muestras aportadas y evaluar el modelo global mediante un conjunto de validación, registrando *Accuracy* y *Loss*.
2. **Clientes Móviles Android (Hardware Real):** Representan los nodos de cómputo en el borde de la red (*Edge Devices*). Operan como aplicaciones nativas ejecutadas en dispositivos físicos heterogéneos (gama media y alta). Cada terminal gestiona de forma aislada sus recursos de procesador y memoria RAM, ejecutando el entrenamiento local mediante LiteRT con retropropagación *on-device* sobre lotes privados de datos locales (dataset MNIST) protegiendo los datos locales. Transmiten los pesos del modelo mediante buffers binarios eficientes.
3. **Clientes Simulados (Scripts Python):** Agentes autónomos ejecutados en entorno de PC que emulan la presencia de nodos móviles adicionales. Esta abstracción permite escalar numéricamente la cantidad de nodos participantes para aislar variables telemáticas y evaluar el comportamiento de la red bajo cargas masivas sin depender exclusivamente de infraestructura física.

La arquitectura garantiza un desacoplamiento estricto entre el cálculo local y el servidor. Las entidades periféricas desconocen el estado de los demás clientes y únicamente interactúan de forma unidireccional con la API pública del servidor central.

3.2. Protocolo de Comunicación y API REST

Para posibilitar el intercambio de información entre el servidor en Python y la diversidad de clientes (desarrollados en Python o Kotlin), se ha implementado un protocolo de aplicación basado en el estilo arquitectónico REST (del inglés *Representational State Transfer*) sobre la pila de protocolos TCP/IP y transporte HTTP.

La API REST del servidor central expone cinco *endpoints* principales diseñados de forma semántica para controlar el flujo federado [14]:

- **GET /config:** Endpoint de sincronización y sondeo. Devuelve un objeto en formato JSON (*JavaScript Object Notation*) con las variables de estado del servidor: la ronda de comunicación actual, el número de épocas locales a entrenar, el identificador único del experimento, las rondas objetivo y el flag de participación.
- **GET /model/download:** Endpoint de descarga para clientes Python. Transmite el modelo global actualizado de la ronda en curso encapsulado como un archivo binario comprimido en formato .npz.
- **GET /model/download/bin:** Endpoint de descarga optimizado para clientes Android. Transmite los pesos y sesgos del modelo global serializados como un flujo de bytes binario crudo (.bin), facilitando la ingesta directa en la memoria RAM del móvil sin sobrecarga de descompresión.
- **POST /model/upload:** Endpoint asíncrono para la recepción de actualizaciones locales. Permite a los clientes subir sus nuevos pesos tras completar el entrenamiento local. Los datos se transmiten mediante una petición multiparte (multipart/form-data), lo que permite empaquetar en una misma solicitud HTTP el archivo binario del modelo junto con los metadatos esenciales en formato texto (ID de cliente, número de muestras procesadas, ronda del cliente y tiempo de entrenamiento local).
- **POST /admin/configurar:** Endpoint de control administrativo que permite la ejecución secuencial de baterías de pruebas. En lugar de requerir un

reinicio manual del servicio web para cada experimento, este endpoint recibe una carga útil en JSON que reinicializa el entorno del servidor: destruye el modelo anterior, inicializa los pesos, establece la semilla estadística para garantizar la reproducibilidad y fija las reglas del nuevo escenario (épocas y rondas), desencadenando así el inicio de una nueva prueba empírica en la flota de dispositivos.

Diagrama de Secuencia del Ciclo de Vida de una Ronda

El siguiente diagrama (véase la Figura 3.2.1) detalla la interacción telemática y la sincronización entre los clientes y el servidor durante el transcurso de una ronda federada completa:

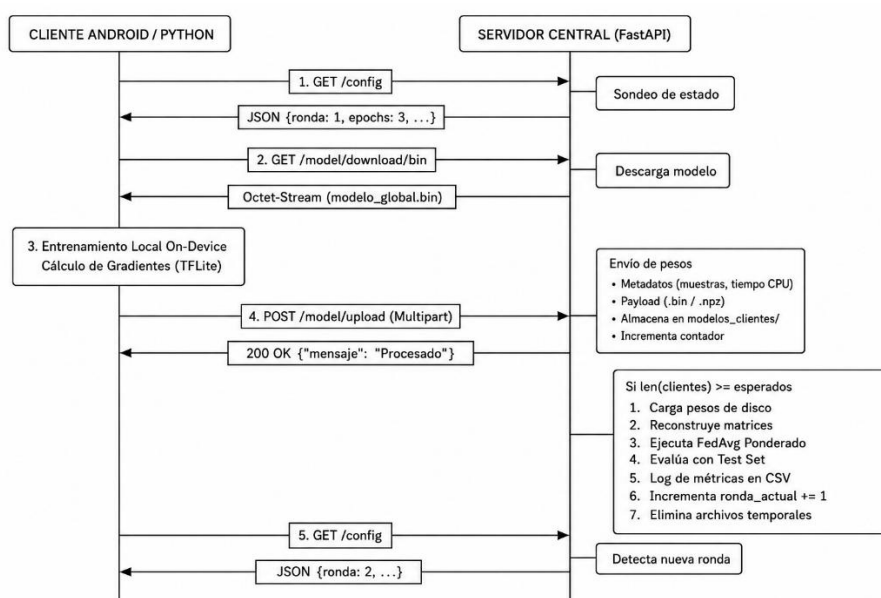


Figura 3.2.1: Ciclo de Vida de una Ronda

La Figura 3.2.1 ilustra el diagrama de secuencia del ciclo de vida estándar de una ronda de Aprendizaje Federado, detallando la interacción telemática entre un cliente periférico (Android o Python) y el Servidor Central (FastAPI). El flujo de ejecución asíncrono se estructura en las siguientes fases secuenciales:

1. Sondeo de Estado (Polling): El ciclo se inicia cuando el cliente realiza una petición HTTP GET al endpoint /config. El servidor central responde con un objeto JSON que contiene los metadatos del estado actual (número de ronda

vigente, épocas locales a entrenar, etc.). El cliente utiliza esta información para determinar si debe iniciar el cómputo o permanecer en reposo.

2. Descarga del Modelo Global: Al detectar que el servidor ha avanzado a una nueva ronda, el cliente solicita los pesos del modelo maestro mediante una petición GET a `/model/download/bin` (o `/model/download` en el caso de agentes Python). El servidor transmite los tensores como un flujo binario crudo (Octet-Stream), minimizando la sobrecarga de red.

3. Entrenamiento Local (*On-Device*): El cliente se desconecta temporalmente del servidor y ejecuta la retropropagación en su propio procesador (mediante TensorFlow Lite en el caso de Android), optimizando el modelo sobre sus datos privados locales durante el número de épocas estipulado.

4. Transmisión de Resultados (*Upload*): Tras finalizar el cálculo de gradientes, el cliente encapsula los nuevos pesos y sus metadatos en una petición HTTP POST de tipo multiparte (`multipart/form-data`) dirigida a `/model/upload`. El servidor almacena temporalmente la carga útil y responde con un código de éxito 200 OK.

5. Fase de Agregación Centralizada: De forma concurrente, el servidor evalúa si el número de clientes que han subido su modelo alcanza el umbral mínimo esperado para la ronda. Al cumplirse esta condición, el servidor bloquea la recepción, carga los pesos del disco, reconstruye las matrices y ejecuta el algoritmo matemático de promediado federado ponderado (FedAvg). A continuación, evalúa el modelo global frente a un conjunto de prueba, registra las métricas en un archivo CSV, incrementa el contador de la ronda y elimina los archivos temporales de los clientes.

6. Reanudación del Ciclo: El cliente, que continúa su bucle de sondeo, realiza un nuevo GET `/config`, detecta el incremento en el contador de ronda y reinicia el proceso.

El *endpoint* POST `/admin/configurar` no aparece en este diagrama porque no forma parte del protocolo de comunicación del Aprendizaje Federado ni del ciclo de vida de los clientes. Este *endpoint* solo es invocado por el script de

automatización antes de que el ciclo comience o entre experimentos distintos para limpiar la memoria del servidor, resetear las rondas a 0 y fijar las semillas estadísticas (seeds).

3.3. Interoperabilidad y Formato de Datos

Uno de los principales retos de ingeniería en este proyecto reside en la heterogeneidad del entorno: coordinar un servidor basado en NumPy/TensorFlow en Python con aplicaciones nativas en Kotlin sobre arquitectura ARM [15]. En las fases iniciales del proyecto, la transmisión de pesos se realizaba mediante el formato comprimido estándar de NumPy (.npz). Aunque este formato es idóneo para entornos puros de Python gracias a su capacidad para estructurar múltiples arrays con metadatos de claves y cabeceras de compresión, introdujo un bloqueo crítico de rendimiento en la aplicación Android. Para resolver la incompatibilidad de serialización de arrays multidimensionales, se han diseñado dos estrategias diferenciadas de transporte de pesos en red:

1. **Formato NumPy Zip (.npz):** Utilizado para el canal de comunicación con los clientes Python. Empaqueta y comprime de forma nativa la lista de matrices de Keras/NumPy, ofreciendo una alta densidad de información en disco y red.
2. **Formato Binario Crudo *Little-Endian* (.bin):** La máquina virtual de Java/Kotlin en Android carece de soporte nativo para deserializar los contenedores .npz de NumPy de forma eficiente. Para resolver esta limitación, se diseñó una "Función Puente" que convierte la memoria RAM de las matrices Float32/Float16 a una secuencia de bytes puros alineados bajo la ordenación *Little-Endian* (estándar adoptado por los procesadores ARM y x86) [16].

El servidor efectúa la reconstrucción de la geometría del modelo interpretando el desplazamiento exacto (*offset*) de cada capa en el buffer de bytes entrante, aislando los paros de ejecución (*crashes*) y garantizando una agregación FedAvg transparente e independiente de la plataforma que originó los datos.

3.4. Control y Sincronización

El ciclo de aprendizaje federado es inherentemente de naturaleza síncrona por rondas, pero opera sobre un canal de red asíncrono y no confiable [17]. Para coordinar a los clientes sin saturar el tráfico ni congelar las interfaces de usuario móviles, se ha implementado una máquina de estados finitos fundamentada en tres pilares:

- **Sondeo Asíncrono (*Polling*):** Los clientes ejecutan un bucle en segundo plano (usando corrutinas en Android sobre el hilo de red Dispatchers.IO y bucles controlados con temporizadores en Python). Periódicamente (cada 3 a 5 segundos), consultan la ruta /config para conocer el estado del servidor. Si el servidor permanece en la misma ronda o en reposo, el cliente no realiza descargas redundantes, entrando en estado inactivo para preservar la batería y el ancho de banda.
- **Gestión de Estados de Ronda:** Cuando la respuesta del servidor indica una ronda superior a la última completada por el cliente (`ronda_servidor > ronda_completada`), el agente activa su estado de entrenamiento, descarga el modelo actualizado, entrena sus datos locales y transmite los resultados.
- **Máquina de Estados del Servidor y Control de Rondas Obsoletas:** El servidor mantiene en memoria un estado global centralizado. Para prevenir ataques de desincronización o el impacto de retardos en la red radio (donde un móvil lento entrega sus resultados cuando el servidor ya ha avanzado a la ronda siguiente), se implementa un filtro de seguridad estricto en el endpoint /model/upload

(ronda_cliente != estado_servidor["ronda_actual"])

Si un cliente intenta subir una actualización perteneciente a una ronda anterior, el servidor la descarta inmediatamente, impidiendo la contaminación del modelo global actualizado.

- **Parada Temprana (*Early Stopping*) y Control por Flag (*rondas_objetivo*):** En entornos de laboratorio, dejar que los clientes

entrenen indefinidamente genera ejecuciones fantasma que consumen recursos de procesador y contaminan los logs de telemetría.

Para solucionar esto, el servidor monitoriza el progreso del modelo. Cuando la precisión (*Accuracy*) alcanza un umbral deseado (por ejemplo, el 90% o 92.5%) o se cumple el límite estricto fijado por la variable *rondas_objetivo*, el servidor conmuta dicho flag a 0. Esta señal ordena inmediatamente a toda la flota distribuida entrar en estado de reposo absoluto, concluyendo el experimento de forma limpia y reproducible.

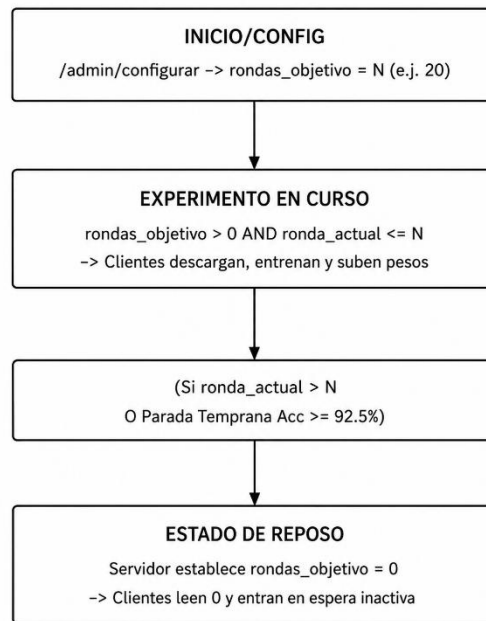


Figura 3.4.1: Ciclo Early Stopping y Rondas_Objetivos

Capítulo 4

Implementación del Sistema

Contenido

4.1.	Servidor Central de Agregación (FastAPI).....	24
4.1.1.	Ciclo de Vida y Recepción de Modelos.....	25
4.1.2.	Control de Rondas Obsoletas y Robustez Temporal	26
4.1.3.	Algoritmo de Agregación Ponderada (FedAvg).....	27
4.1.4.	Subsistema de Telemetría y Exportación de Logs.....	28
4.2.	Cliente Android (Kotlin + TFLite/LiteRT)	29
4.2.1.	Arquitectura de la App y Módulo	31
4.2.2.	Solución a Retos Técnicos de Ingeniería.....	32
4.3.	Herramientas de Automatización	34
4.3.1.	Orquestador de Experimentos (<i>ejecutar_experimentos.py</i>)	35
4.3.2.	Módulo de Post-procesamiento y Generación de Gráficas (<i>generar_graficas.py</i>)	36

4.1. Servidor Central de Agregación (FastAPI)

El servidor central asume la responsabilidad de coordinar matemáticamente y gestionar telemáticamente la red federada. Su desarrollo se ha sustentado sobre el *framework* asíncrono FastAPI, apoyado en el servidor ASGI (*Asynchronous Server Gateway Interface*) Uvicorn. Esta combinación tecnológica permite operar sobre un bucle de eventos no bloqueante mediante corrutinas nativas, asegurando una atención simultánea y de baja latencia frente a múltiples

conexiones entrantes de clientes sin incurrir en la sobrecarga de memoria asociada a los modelos clásicos basados en hilos concurrentes bloqueantes [18].

En la Figura 4.1.1 se muestra los componentes del servidor y su interacción y en las próximas secciones se describirán en detalle.

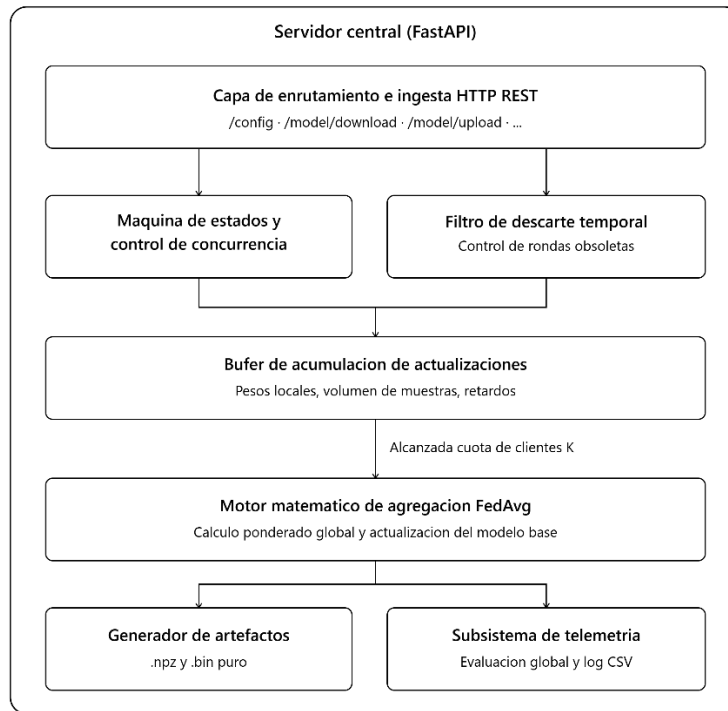


Figura 4.1.1: Esquema Servidor ASGI con Uvicorn

4.1.1. Ciclo de Vida y Recepción de Modelos

El servidor administra el ciclo de vida de cada ronda federada a través de un esquema de estado global persistido en memoria. Cuando un nodo periférico completa su fase de optimización local, empaqueta los parámetros actualizados y efectúa una llamada HTTP de tipo POST hacia el punto de acceso /model/upload.

La ingesta de estos datos se realiza mediante un esquema de transporte multiparte (multipart/form-data). Esta aproximación permite transmitir de manera atómica tanto la carga binaria (*payload*) con los pesos neuronales como un

cuerpo complementario de metadatos estructurados en texto plano. Entre estos parámetros destacan:

- El identificador único del dispositivo emisor (*client_id*).
- El índice de la ronda federada a la que corresponde el entrenamiento (*client_round*).
- La cantidad exacta de muestras locales computadas durante el ciclo (n_k).
- El tiempo de procesamiento local invertido por la CPU o procesador del cliente (t_{train}).

Al recepcionar la solicitud, el servidor extrae la carga útil, valida la integridad de los metadatos y deposita los parámetros en una estructura interna de acumulación temporal, dejando el socket liberado inmediatamente para atender a los siguientes participantes.

4.1.2. Control de Rondas Obsoletas y Robustez Temporal

En un entorno telemático real donde operan terminales móviles a través de redes inalámbricas, las fluctuaciones de cobertura, las retransmisiones por pérdida de paquetes y la disparidad en las capacidades de cálculo pueden ocasionar que un dispositivo complete y envíe su actualización con retraso, cuando el servidor ya ha avanzado a una ronda federada superior. La incorporación de estos parámetros desactualizados en la agregación distorsionaría la trayectoria de convergencia matemática del modelo global.

Para solventar esta contingencia, se ha integrado una barrera lógica de validación temporal en la cabecera de la rutina de ingesta. El servidor compara el valor *client_round* reportado en la petición con el contador global *current_round* vigente. Si se detecta una discrepancia temporal (esto es, si el cliente remite una actualización perteneciente a un ciclo previo), el servidor aborta el procesamiento de la carga útil, registra un evento de descarte por obsolescencia en su subsistema de registro (*logging*) y emite una respuesta HTTP *400 Bad Request*. De este modo se previene la corrupción del búfer de

agregación y se salvaguarda la estabilidad de la red sin comprometer la disponibilidad del servicio.

4.1.3. Algoritmo de Agregación Ponderada (FedAvg)

Cuando el búfer de recepción alcanza la cuota mínima de K clientes requerida para la ronda en curso, el servidor bloquea temporalmente la admisión de nuevas cargas y desencadena el proceso de cálculo de FedAvg. Este valor K no es calculado dinámicamente en tiempo de ejecución, sino que actúa como un hiperparámetro de configuración determinista inyectado en el servidor central al inicio de cada batería de pruebas a través del *endpoint* de control (/admin/configurar).

De acuerdo con el formalismo matemático del algoritmo, la contribución de cada cliente al nuevo modelo global no es uniforme, sino directamente proporcional a la fracción de datos locales que dicho nodo aporta al conjunto total computado en esa iteración. Matemáticamente, el proceso se modela mediante la siguiente expresión:

$$w_{t+1} = \sum_{k=1}^K \frac{n_k}{\sum_{j=1}^K n_j} \times w_t^{(k)} \quad (4.1)$$

donde w_{t+1} simboliza el conjunto de parámetros (matrices de pesos y vectores de sesgo) del nuevo modelo global resultante, $w_t^{(k)}$ corresponde a los tensores locales enviados por el cliente k tras la ronda t , y n_k denota el número de muestras procesadas por dicho terminal.

La dependencia estricta de este umbral K expone al sistema a bloqueos (*Deadlocks*) inherentes a las redes síncronas. Si un dispositivo pierde la cobertura o activa una simulación de pérdida de red (*Dropout*), el búfer de recepción quedaría estancado indefinidamente en $K - 1$ actualizaciones, congelando el avance del modelo global. Aunque en este entorno controlado la

estabilidad se garantiza mediante el orquestador automatizado, en despliegues de producción reales esta limitación exige implementar temporizadores de ronda (*timeouts*) que fuercen una agregación parcial con los nodos disponibles si no se alcanza la cuota K en un tiempo límite.

A nivel computacional, el servidor itera de forma secuencial sobre cada capa de la red neuronal. Para cada capa, realiza una combinación lineal ponderada multiplicando cada tensor local por el factor de escala normalizado n_k/n_{total} y acumulando los resultados en un tensor global maestro. Una vez finalizado el cálculo para toda la topología, el nuevo estado de pesos sobrescribe la memoria del modelo base, se incrementa el contador de ronda ($t \leftarrow t + 1$), se vacía el búfer temporal y se desbloquea el sistema para habilitar la descarga de la nueva versión.

4.1.4. Subsistema de Telemetría y Exportación de Logs

Con el fin de analizar el comportamiento del sistema con rigor telemático y experimental, el servidor incorpora un módulo de monitorización y evaluación automatizada que actúa inmediatamente después de cada agregación.

En primer lugar, el servidor ejecuta una fase de validación centralizada. Para ello, evalúa el modelo global recién actualizado sobre un conjunto de datos de testeo estandarizado e independiente (compuesto por 10.000 muestras del conjunto MNIST), extrayendo métricas de pérdida (*Loss*) y de precisión predictiva (*Accuracy*). En paralelo, consolida los metadatos telemáticos recopilados de los clientes en esa ronda, calculando el tiempo medio de cómputo periférico, el tiempo total invertido por la ronda y el volumen acumulado de tráfico de red transferido.

Toda esta información se vuelca secuencialmente en un archivo de registro en formato .csv (*Comma-Separated Values*). La estructura del fichero organiza cronológicamente las columnas correspondientes al número de ronda, la precisión global obtenida, el valor de pérdida, la latencia media de cómputo en

los clientes, el tráfico generado en kilobites (KB) y el número de participantes efectivos. Este archivo estructurado garantiza la trazabilidad del experimento y facilita la posterior generación de gráficas telemáticas y curvas de aprendizaje.

4.2. Cliente Android (Kotlin + TFLite/LiteRT)

El desarrollo del nodo móvil se ha realizado de manera nativa para el sistema operativo Android utilizando el lenguaje de programación Kotlin. La gran mayoría de los marcos de trabajo de inteligencia artificial para dispositivos móviles están concebidos exclusivamente para inferencia (*Forward Pass*). Implementar el ciclo completo de entrenamiento supervisado que incluya la retropropagación de errores (*Backward Pass*) directamente en el procesador del terminal exige una arquitectura interna adaptada a las restricciones de memoria y cómputo de la arquitectura ARM [19].

Para dar respuesta a estas exigencias técnicas, se ha diseñado la topología de *software* que se ilustra en la Figura 4.2.1. El flujo de control y procesamiento de la aplicación se encuentra fuertemente desacoplado en las siguientes capas funcionales:

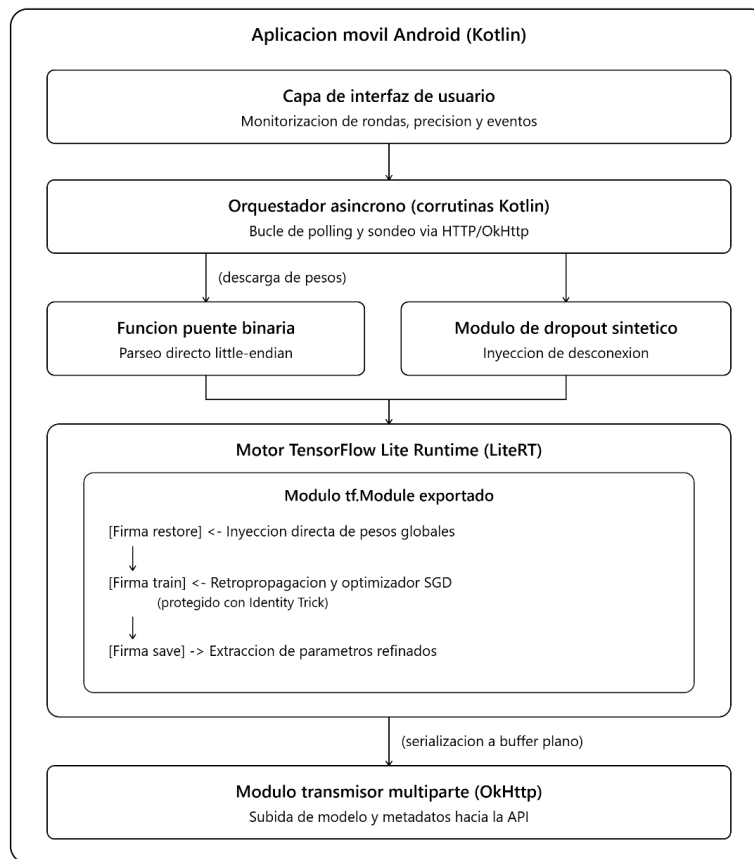


Figura 4.2.1: Arquitectura app móvil android LiteRT

- **Capa de Interfaz de Usuario:** Responsable de la monitorización en tiempo real del progreso de las rondas, la precisión de clasificación y los eventos telemáticos. Opera sobre el hilo principal (*Main Thread*) sin sufrir bloqueos durante el cómputo.
- **Orquestador Asíncrono:** Implementado mediante corrutinas de Kotlin, gestiona el bucle de sondeo (*polling*) en segundo plano utilizando el cliente HTTP OkHttp. Su función es interrogar al servidor y coordinar el instante exacto en el que debe iniciarse la descarga de pesos.
- **Acondicionamiento y Simulación:** Una vez que los pesos se descargan, el sistema delega la ingesta a una Función puente binaria, encargada de realizar un parseo directo en memoria aplicando la convención *Little-Endian*.
- **Motor TensorFlow Lite Runtime (LiteRT):** Constituye el núcleo de ejecución local. A diferencia de las redes estáticas de inferencia, este motor

interpreta un módulo `tf.Module` exportado desde Python que expone tres firmas computacionales (*signatures*) diferenciadas:

- **Firma restore:** Permite la inyección directa y eficiente de los pesos globales recién descargados en las variables mutables del modelo local.
 - **Firma train:** Ejecuta el paso hacia adelante, la retropropagación de gradientes y la actualización mediante el optimizador SGD.
 - **Firma save:** Tras finalizar las épocas de entrenamiento, esta firma se encarga de la extracción de los parámetros ya refinados.
- **Módulo Transmisor Multiparte:** Finalmente, los tensores extraídos se someten a una serialización hacia un *buffer* plano de bytes. Este módulo emplea la librería `OkHttp` para empaquetar el archivo binario resultante junto con los metadatos temporales de la ronda, procediendo a su subida final hacia la API REST del servidor central.

4.2.1. Arquitectura de la App y Módulo

Para hacer viable el entrenamiento en el propio terminal (*on-device*), se diseñó la red neuronal como un módulo de TensorFlow derivado de la clase base `tf.Module`. Este contenedor se compila mediante TensorFlow Lite Converter integrando variables mutables que representan los pesos y sesgos de las capas densas, y exponiendo tres firmas computacionales (*signatures*) diferenciadas:

- **Firma restore:** Recibe como argumentos de entrada tensores con los pesos globales descargados de la red y los inyecta de forma directa sobre las variables mutables internas de la red, sobrescribiendo el estado local del modelo sin necesidad de reconstruir la sesión.
- **Firma train:** Recibe matrices correspondientes a un lote de datos locales de entrenamiento junto con sus etiquetas de clasificación. Ejecuta la propagación hacia adelante, evalúa la función de coste (*Cross-Entropy*), calcula los gradientes de error mediante diferenciación automática y

actualiza los parámetros empleando un optimizador de descenso de gradiente estocástico (SGD).

- **Firma save:** Lee el estado de los tensores internos tras completar las épocas de entrenamiento local y los devuelve como arreglos numéricos planos para su posterior empaquetado y transmisión.

Dentro de la aplicación Android, este grafo compilado es ejecutado por el intérprete LiteRT. La orquestación general se gestiona mediante corrutinas de Kotlin enlazadas al despachador de operaciones de entrada y salida (Dispatchers.IO), asegurando que ni las operaciones matriciales intensivas ni las transferencias telemáticas bloqueen el hilo principal de renderizado de la interfaz gráfica (*UI Thread*).

4.2.2. Solución a Retos Técnicos de Ingeniería

Durante la integración del cliente móvil surgieron diversos obstáculos telemáticos y de compilación que requirieron el diseño de soluciones de ingeniería específicas:

A. El "Identity Trick" frente a la Eliminación de Código Muerto (*Dead Code Elimination*)

Durante el proceso de conversión del modelo de TensorFlow a su formato ejecutable ligero TFLite, el compilador aplica por defecto algoritmos de optimización estática de grafos, entre los cuales destaca la eliminación de código muerto (*Dead Code Elimination*, DCE) [20].

Dado que la firma train está orientada a mutar variables internas (pesos sinápticos) y habitualmente no devuelve un flujo de datos explícito hacia la salida, el optimizador interpreta erróneamente que la función carece de efectos colaterales observables. Como consecuencia de este análisis estático, el compilador poda el árbol de operaciones de cálculo de gradientes y actualización de variables, generando un archivo binario final que únicamente es capaz de inferir, pero no de entrenar.

Para eludir este comportamiento sin alterar el algoritmo de entrenamiento, se concibió la técnica denominada *Identity Trick* (o salida dummy). La solución radica en forzar una dependencia de datos sintética dentro del grafo: se toma el escalar resultante de la función de pérdida (*Loss*) y se envuelve mediante una operación de identidad matemática (`tf.identity`), asignándolo formalmente como valor de retorno de la firma `train`. Al existir una dependencia directa entre la salida del grafo y el cálculo de la pérdida, el compilador se ve forzado a preservar íntegramente la cadena de operaciones de retropropagación y derivación automática, garantizando la funcionalidad completa del entrenamiento en el dispositivo móvil.

B. La Función Puente Binaria (*Little-Endian*)

Un reto crítico de interoperabilidad radicaba en la discrepancia de formatos entre plataformas: el servidor opera con bibliotecas científicas de Python basadas en NumPy (que empaquetan matrices en ficheros comprimidos `.npz`), mientras que el entorno de ejecución Android (máquina virtual de Java/ART) carece de soporte nativo ligero para decodificar estas estructuras de forma eficiente.

Para superar este cuello de botella sin degradar los recursos del móvil mediante librerías de descompresión pesadas, se diseñó una función puente de bajo nivel. Esta función mapea las matrices numéricas multidimensionales directamente a un búfer continuo de bytes sin cabeceras propietarias, aplicando de forma estricta la ordenación *Little-Endian* (`ByteOrder.LITTLE_ENDIAN`), correspondiente a la arquitectura nativa de los procesadores ARM y x86.

El cliente Android lee secuencialmente los segmentos de memoria mediante punteros calculados a partir de la topología conocida de la red (número de entradas, dimensiones de las capas ocultas y clases de salida), inyectando los valores directamente en la firma `restore`. De este modo, la deserialización se ejecuta con coste computacional y de memoria prácticamente nulo.

4.3. Herramientas de Automatización

Para garantizar la reproducibilidad científica del estudio, eliminar la variabilidad derivada de la intervención manual y facilitar el análisis masivo de métricas, se ha desarrollado un ecosistema de automatización estructurado. La Figura 4.3.1 ilustra este flujo de automatización y análisis, el cual se compone de las siguientes cuatro etapas consecutivas:

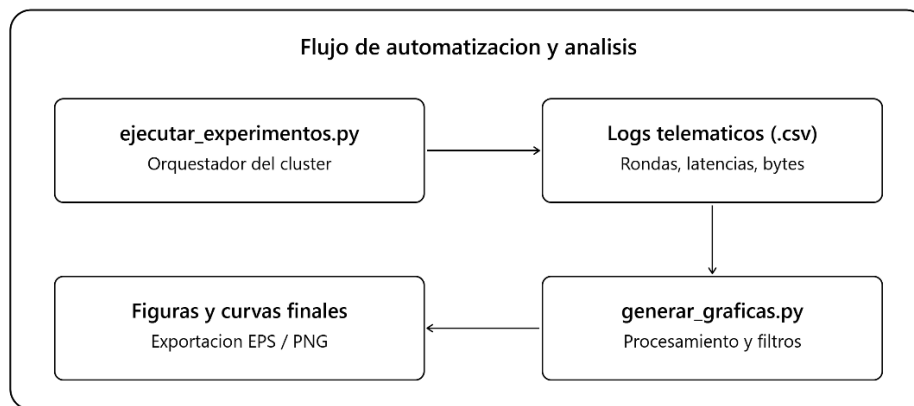


Figura 4.3.1: Flujo de Automatización y Análisis

- **ejecutar_experimentos.py (Orquestador del clúster):** Script maestro encargado de configurar, lanzar y supervisar automáticamente las baterías de pruebas sobre la flota de clientes.
- **Logs telemáticos (.csv):** Archivos de registro que almacenan de forma estructurada las métricas en crudo resultantes de la ejecución (rondas, latencias y volumen de bytes).
- **generar_graficas.py (Procesamiento y filtros):** Herramienta analítica que ingiere los ficheros de registro para consolidar los datos, agrupar las semillas estadísticas y aplicar filtros matemáticos.
- **Figuras y curvas finales:** Resultado visual del procesamiento, exportado en formatos de alta resolución (EPS o PNG) para su evaluación directa en la memoria.

A continuación, se detalla en profundidad el diseño y la lógica de ingeniería subyacente a los dos módulos principales de procesamiento en Python.

4.3.1. Orquestador de Experimentos (*ejecutar_experimentos.py*)

Este módulo actúa como el controlador maestro del laboratorio distribuido. Su cometido es coordinar la ejecución secuencial de decenas de configuraciones experimentales de forma completamente desatendida. La Figura 4.3.2 detalla la arquitectura interna de este orquestador, cuyo flujo operativo se articula a partir de una matriz inicial y cuatro fases de ejecución:

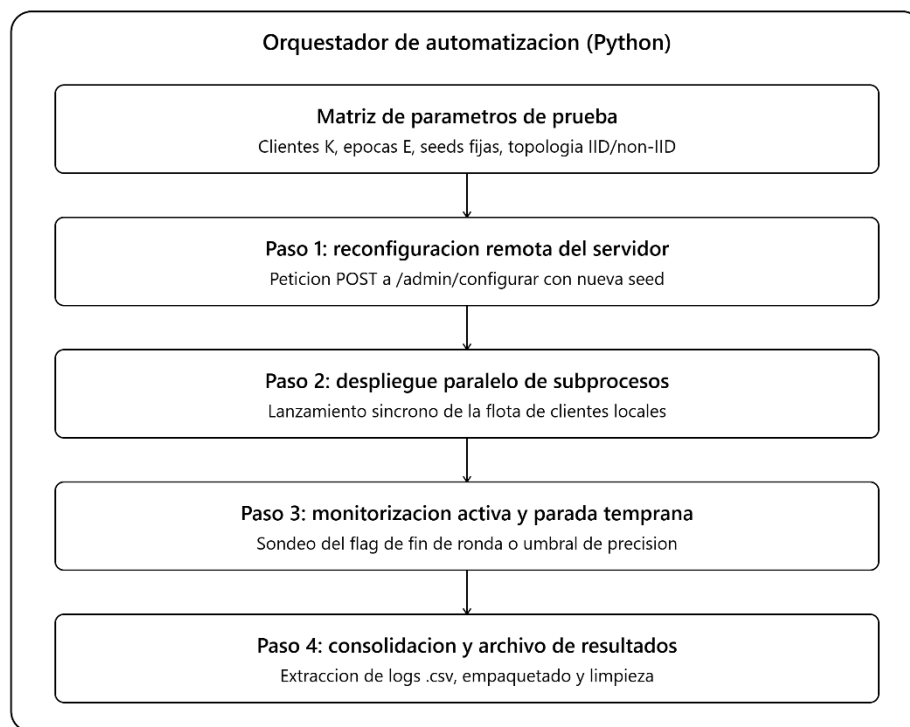


Figura 4.3.2: Flujo de automatización de experimentos

1. **Lectura y Configuración de Parámetros:** Carga la matriz de pruebas definida para la batería experimental, extrayendo las variables a evaluar: número total de clientes participantes, número de épocas locales (E), distribución estadística asignada (IID o Non-IID) y semillas pseudoaleatorias de control (seed).
2. **Sincronización y Reinicio Remoto:** Realiza una llamada HTTP POST hacia el *endpoint* de administración `/admin/configurar` del servidor central. Esta acción restablece el modelo global a sus condiciones iniciales,

actualiza los parámetros telemáticos y prepara el búfer de agregación para el nuevo test.

3. **Despliegue y Supervisión de Nodos:** Instancia de manera concurrente los procesos correspondientes a la flota de clientes periféricos, monitorizando la evolución de las rondas a través de sondeos no invasivos.
4. **Cierre y Persistencia Automatizada:** Al detectarse el cumplimiento del criterio de parada (por haber alcanzado el número máximo de rondas o el umbral de precisión definido), el orquestador ordena la detención limpia de los procesos, consolida los archivos de métricas .csv generados y los cataloga en el directorio de resultados para su posterior análisis.

4.3.2. Módulo de Post-procesamiento y Generación de Gráficas **(*generar_graficas.py*)**

La ingesta masiva de datos durante los ensayos da lugar a colecciones extensas de ficheros .csv con registros temporales y métricas telemáticas. El script *generar_graficas.py* se encarga de transformar estos datos brutos en representaciones visuales estandarizadas aptas para el análisis técnico [21].

- **Consolidación Estadística y Agrupación:** Agrupa las repeticiones ejecutadas bajo distintas semillas aleatorias (seed) para una misma configuración experimental. Calcula de forma automática la media muestral y la desviación estándar de la precisión (*Accuracy*), la pérdida (*Loss*) y el retardo por ronda, permitiendo representar curvas con intervalos de confianza que aíslan el efecto de la estocasticidad.
- **Cómputo Acumulado de Tráfico de Red:** Procesa las columnas de bytes transmitidos por subida y bajada, calculando el tráfico agregado en megabytes (MB) en función del número de rondas y el esquema de serialización empleado (archivos .npz frente a binarios crudos .bin o cuantizados a Float16).

- **Exportación Gráfica Estandarizada:** Utiliza librerías de representación científica (Matplotlib) para generar figuras vectoriales de alta resolución. El módulo parametriza uniformemente las tipografías, el etiquetado de ejes con sus respectivas magnitudes y unidades, y los estilos de trazo, asegurando la coherencia visual exigida en memorias académicas [22].

Gracias a este diseño, todas las baterías de ensayos experimentales se ejecutan bajo condiciones idénticas e independientes, garantizando la reproducibilidad estadística de los datos, además de encargarse de forma automática del post-procesamiento de datos y la generación de gráficos.

Parte III

Experimentación

Capítulo 5

Plan de Experimentación y Análisis de Resultados

Contenido

5.1.	Metodología de Evaluación.....	42
5.1.1.	Parámetros Configurables del Sistema.....	43
5.1.2.	Métricas Telemáticas y de Rendimiento Registradas	43
5.1.3.	Validez Estadística mediante Control de Semillas	44
5.2.	Diseño del Bloque 1: Hardware Real y Cuellos de Botella Físicos	45
5.2.1.	Experimento 1.1: Validación del Entrenamiento Real <i>On-Device</i> en Cliente Android Solitario.....	46
5.2.2.	Experimento 1.2: Aislamiento y Cuantificación del Efecto Rezagado (<i>Straggler Effect</i>).....	47
5.3.	Diseño del Bloque 2: Simulaciones a Escala y Eficiencia de Red	47
5.3.1.	Experimento 2.1: Escenarios de Distribución de Datos (IID vs. Non-IID Patológico).....	49
5.3.2.	Experimento 2.2: Compromiso entre Cómputo Local y Comunicación (<i>Trade-off</i>).....	49
5.3.3.	Experimento 2.3: Evaluación Comparativa de Técnicas de Optimización Telemática	50
5.3.4.	Experimento 2.4: Escalabilidad del Tráfico de Red frente al Aumento de Nodos	50
5.4.	Diseño del Bloque 3: Flotas Mixtas y Agregación Desbalanceada	51
5.4.1.	Experimento 3.1: Flota Híbrida Equilibrada (Proporción 1:1).....	52

5.4.2. Experimento 3.2: Flotas Híbridas Desbalanceadas (Proporciones 1:10 y 1:50)	53
---	----

5.1. Metodología de Evaluación

La evaluación del sistema federado se aborda desde una perspectiva telemática y de sistemas distribuidos, priorizando la eficiencia del canal de comunicación y el comportamiento temporal del clúster frente a la mera optimización algorítmica. Para estructurar este análisis empírico, se ha definido el modelo de evaluación ilustrado en la Figura 5.1.1, el cual muestra el flujo experimental en tres bloques secuenciales:

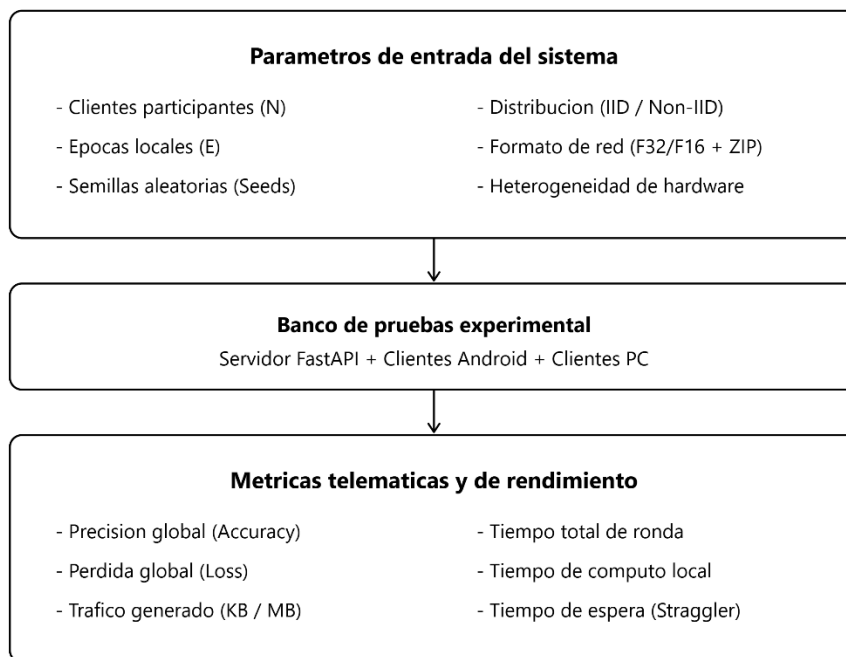


Figura 5.1.1: Parámetros de Evaluación para pruebas de Experimentación

El flujo experimental parte de la configuración de los parámetros de entrada (como el número de clientes, distribución de datos o formato de red), los cuales

se ejecutan sobre el banco de pruebas híbrido (servidor central, dispositivos móviles y simulados) para extraer finalmente las correspondientes métricas telemáticas y de rendimiento (precisión, latencias y volumen de tráfico).

A continuación, se detalla en profundidad cada uno de estos elementos.

5.1.1. Parámetros Configurables del Sistema

El comportamiento de la red distribuida depende de un conjunto de variables de control que se manipulan de forma aislada en cada batería de ensayos:

- **Escala de Participación:** Número de clientes totales esperados por ronda (`clientes_esperados`).
- **Hiperparámetros de Entrenamiento Local:** Épocas locales (`epochs_locales`) y tamaño del *batch*.
- **Distribución de Datos:** Cantidad de muestras por cliente (`num_muestras`) y grado de heterogeneidad de los datos (IID vs. Non-IID).
- **Eficiencia de Comunicación:** Nivel de precisión de los pesos (cuantización a Float32 vs Float16) y algoritmo de compresión (`np.savez_compressed` vs `.npz` estándar / `.bin` crudo).
- **Naturaleza del Cliente:** Hardware real (dispositivo Android) vs. Simuladores lógicos (scripts en Python).

5.1.2. Métricas Telemáticas y de Rendimiento Registradas

Para cuantificar el desempeño global del sistema, el módulo de telemetría almacena de manera síncrona las siguientes magnitudes en los ficheros de registro `.csv`:

RENDIMIENTO DEL MODELO

- **Accuracy Global (%):** Precisión del modelo tras la agregación en el servidor.

- **Loss Global (%):** Función de pérdida que indica el margen de error del modelo.
- **Velocidad de Convergencia:** Número de rondas necesarias para alcanzar un Accuracy objetivo (ej. 90%).

RENDIMIENTO DE RED Y HARDWARE

- **Tráfico Total por Ronda (KB):** Suma del tamaño de subida y bajada de todos los clientes.
- **Tráfico de red generado (KB):** Volumen total de información transmitida a nivel de capa de aplicación, medido en kilobytes.
- **Tiempo Medio de Entrenamiento Local (s):** Tiempo real de reloj transcurrido que el procesador de los dispositivos dedica a la ejecución del entrenamiento.
- **Tiempo Total de Ronda (s):** Intervalo temporal transcurrido desde que el servidor abre la ronda federada hasta que recibe, valida y agrega la totalidad de las actualizaciones requeridas.
- **Efecto Straggler/Espera rezagados (s):** Tiempo en el que el servidor y los clientes rápidos permanecen bloqueados en estado inactivo (*idle*) esperando a que el terminal más lento complete su cálculo y transmisión.

5.1.3. Validez Estadística mediante Control de Semillas

En entornos de *Machine Learning* distribuido, la estocasticidad introducida por la inicialización aleatoria de pesos neuronales, el desordenamiento (*shuffling*) de los datos y el orden de los lotes puede distorsionar las comparaciones telemáticas [23].

Para asegurar la validez científica y permitir la comparación equitativa entre distintas arquitecturas de red, se implementó un protocolo de control estadístico estricto:

- Cada experimento se repite un número prefijado de iteraciones independientes empleando una secuencia de semillas pseudoaleatorias (*seeds*) normalizadas (seed=42, seed=123, etc.).
- La semilla fija de forma determinista la partición del conjunto de datos entre los clientes y el punto inicial de los pesos sinápticos.
- El procesamiento posterior computa la media muestral (μ) y la desviación estándar (σ) para cada punto temporal, permitiendo evaluar el rendimiento global mediante intervalos de confianza.

5.2. Diseño del Bloque 1: Hardware Real y Cuellos de Botella Físicos

El primer bloque experimental tiene como propósito validar la viabilidad técnica del entrenamiento *on-device* en dispositivos móviles reales y analizar empíricamente el impacto de la heterogeneidad del hardware sobre la latencia global del sistema. Tal y como esquematiza la Figura 5.2.1, esta fase se articula en dos ensayos fundamentales:

- Experimento 1.1 (Cliente Android solitario): Validación de la estabilidad computacional del ciclo de entrenamiento y consumo de memoria local utilizando el motor LiteRT.
- Experimento 1.2 (Flota móvil heterogénea): Cuantificación de la degradación temporal en la red (*Straggler Effect*) mediante la confrontación concurrente de dispositivos con capacidades de procesamiento asimétricas (Samsung Galaxy S25 frente a Galaxy M23).

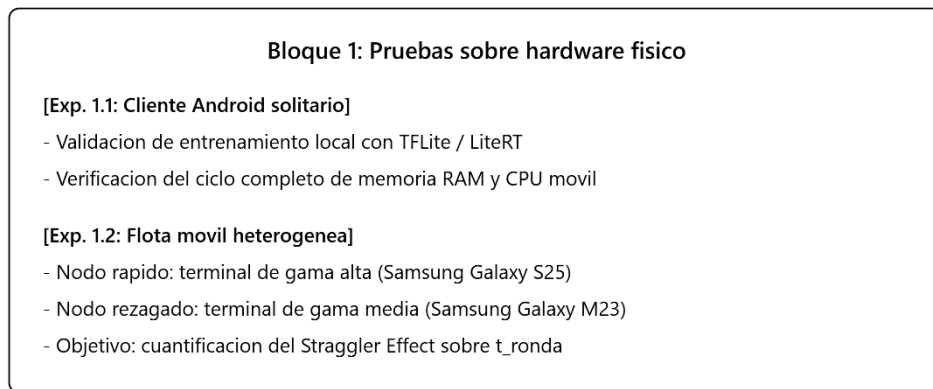


Figura 5.2.1: Experimentos Bloque 1

A continuación, se detalla el diseño metodológico de ambos escenarios.

5.2.1. Experimento 1.1: Validación del Entrenamiento Real *On-Device* en Cliente Android Solitario

- **Validación:** Demostrar que un dispositivo móvil físico es capaz de ejecutar de forma autónoma y estable el ciclo completo de cálculo matricial, optimización de gradientes y serialización de parámetros utilizando la librería LiteRT sin degradar la memoria ni provocar el cierre de la aplicación (finalizaciones incorrectas del sistema).
- **Escenario de Prueba:** Se despliega la aplicación Android en un único terminal móvil conectado a la red local vía Wi-Fi. El servidor central orquesta una secuencia de 10 rondas federadas con una época local ($E=1$) por ronda.
- **Variables Analizadas:** Estabilidad del tráfico de red, integridad de los pesos serializados en la función puente binaria y evolución de la curva de convergencia local (Loss y Accuracy).

5.2.2. Experimento 1.2: Aislamiento y Cuantificación del Efecto Rezagado (*Straggler Effect*)

- **Objetivo:** Aislar el cuello de botella temporal provocado por la disparidad en la capacidad de cómputo de los procesadores móviles heterogéneos dentro de un clúster síncrono.
- **Escenario de Prueba:** Se conforma una red federada física compuesta por dos terminales móviles de especificaciones técnicas asimétricas:
 - Terminal de alta capacidad: Dispositivo móvil de gama alta (Samsung Galaxy S25), equipado con un procesador de alto rendimiento.
 - Terminal de capacidad estándar: Dispositivo móvil de gama media (Samsung Galaxy M23), dotado de una arquitectura de procesamiento más contenida.
- **Hipótesis:** En un protocolo de agregación síncrono como FedAvg, el tiempo total de ronda queda acotado superiormente por el terminal más lento. Este ensayo busca cuantificar el porcentaje exacto de tiempo que el nodo rápido permanece inactivo esperando la finalización del nodo rezagado (*straggler*).

5.3. Diseño del Bloque 2: Simulaciones a Escala y Eficiencia de Red

El segundo bloque experimental recurre a clientes simulados en PC para escalar la flota de participantes, permitiendo aislar variables telemáticas complejas y estudiar la eficiencia en la transmisión de datos a gran escala. Como se puede ver en la Figura 5.3.1, esta fase se divide en cuatro experimentos distintos:

Bloque 2: Simulaciones a escala y eficiencia de red

[Exp. 2.1: Distribucion de datos]

- Escenario IID: muestras balanceadas (clases 0 al 9 uniformes)
- Escenario Non-IID: muestras patologicas (2 clases por cliente)
- Objetivo: evaluar el impacto de la heterogeneidad de datos en la convergencia

[Exp. 2.2: Computo vs. comunicacion]

- Variacion de epocas locales: $E = 1, E = 5, E = 10$
- Objetivo: evaluar la reduccion de rondas y trafico necesario

[Exp. 2.3: Tecnicas de optimizacion telematica]

- F32_RAW vs. F32_ZIP: evaluacion del algoritmo de compresion
- F16_RAW vs. F16_ZIP: evaluacion de cuantizacion + compresion
- Objetivo: cuantificar la reduccion de trafico manteniendo la precision del modelo

[Exp. 2.4: Escalabilidad del trafico de red]

- Variacion de participantes simultaneos: $N = \{2, 5, 10, 20\}$
- Objetivo: determinar el modelo de crecimiento del ancho de banda

Figura 5.3.1: Experimentos Bloque 2

- Experimento 2.1 (Distribución de datos): Evaluación del impacto de la heterogeneidad estadística (IID vs. Non-IID) sobre la convergencia del modelo.
- Experimento 2.2 (Cómputo vs. comunicación): Análisis de la reducción de rondas y tráfico necesario al variar las épocas locales de entrenamiento (E).
- Experimento 2.3 (Optimización telemática): Cuantificación del ahorro de red aplicando algoritmos de compresión y cuantización sin pérdida significativa de precisión.
- Experimento 2.4 (Escalabilidad del tráfico): Determinación del modelo de crecimiento del ancho de banda al incrementar el número de participantes concurrentes (N).

A continuación, se profundiza y con más detalle el diseño de cada uno de los escenarios.

5.3.1. Experimento 2.1: Escenarios de Distribución de Datos (IID vs. Non-IID Patológico)

- **Objetivo:** Analizar el impacto de la dispersión estadística de los datos locales sobre la estabilidad de la agregación federada y el número de rondas requeridas para converger [9].
- **Diseño del Escenario:** Se fija una flota de $N = 5$ clientes y se contrastan dos esquemas de partición sobre el dataset MNIST:
 - Escenario IID: El conjunto de datos se baraja y se divide uniformemente, recibiendo cada nodo muestras balanceadas de las diez clases de dígitos (0 al 9).
 - Escenario Non-IID Patológico: Los datos se ordenan por etiquetas y se fragmentan en fragmentos conjuntos (*shards*), asignando a cada cliente muestras pertenecientes únicamente a 2 clases específicas (por ejemplo, el cliente 1 solo posee imágenes de dígitos 0 y 1).
- **Hipótesis:** La partición Non-IID inducirá divergencia de pesos (*Weight Divergence*), generando oscilaciones en la curva de pérdida y demandando mayor tráfico de red total para alcanzar un umbral de precisión equivalente.

5.3.2. Experimento 2.2: Compromiso entre Cómputo Local y Comunicación (*Trade-off*)

- **Objetivo:** Evaluar cómo varía la carga de la red al alterar el número de épocas locales de entrenamiento ejecutadas por los clientes antes de cada comunicación con el servidor.
- **Diseño del Escenario:** Se ejecuta un conjunto de pruebas fijando $N = 5$ clientes y evaluando tres regímenes de computación local: $E = 1$, $E = 5$ y $E = 10$ épocas locales.
- **Hipótesis:** Incrementar el cómputo local ($E=5$ o $E=10$) reduce drásticamente el número de rondas federadas necesarias para alcanzar

un umbral de precisión objetivo (por ejemplo, $\text{Accuracy} \geq 92.5\%$), disminuyendo el volumen agregado de bytes transmitidos por el enlace de telecomunicaciones a costa de un mayor consumo de batería en los nodos periféricos.

5.3.3. Experimento 2.3: Evaluación Comparativa de Técnicas de Optimización Telemática

- **Objetivo:** Cuantificar el ahorro de ancho de banda obtenido mediante la aplicación combinada de técnicas de cuantización numérica y compresión de datos.
- **Diseño del Escenario:** Se monitoriza el tráfico generado por ronda transmitiendo el modelo bajo cuatro configuraciones telemáticas normalizadas:
 - *F32_RAW*: Parámetros en precisión simple estándar (Float32, 4 bytes por peso) transmitidos en flujo binario directo sin comprimir.
 - *F32_ZIP*: Parámetros en Float32 comprimidos mediante el algoritmo Deflate/ZIP previo al envío.
 - *F16_RAW*: Parámetros cuantizados a media precisión (Float16, 2 bytes por peso) sin compresión.
 - *F16_ZIP*: Parámetros en Float16 sometidos a compresión ZIP.
- **Hipótesis:** La reducción de la precisión aritmética a 16 bits reduce el tamaño del payload al 50%, manteniendo la convergencia matemática con una degradación despreciable de la precisión global.

5.3.4. Experimento 2.4: Escalabilidad del Tráfico de Red frente al Aumento de Nodos

- **Objetivo:** Determinar el modelo de crecimiento del tráfico agregado en el servidor central al escalar la dimensión del clúster federado.

- **Diseño del Escenario:** Se ejecutan pruebas de estrés variando el número de participantes concurrentes en la red: $N = \{2, 5, 10, 20\}$ clientes.
- **Hipótesis:** El tráfico de subida (*uplink*) acumulado por el servidor experimenta un crecimiento estrictamente lineal, lo que evidencia la necesidad de técnicas de selección de clientes o agregación jerárquica en redes con miles de terminales móviles.

5.4. Diseño del Bloque 3: Flotas Mixtas y Agregación Desbalanceada

El tercer, y último, bloque experimental aborda la interacción simultánea entre nodos físicos y simulados bajo condiciones de heterogeneidad en el volumen de datos. Tal y como resume la Figura 5.4.1, esta fase se divide en dos escenarios principales:

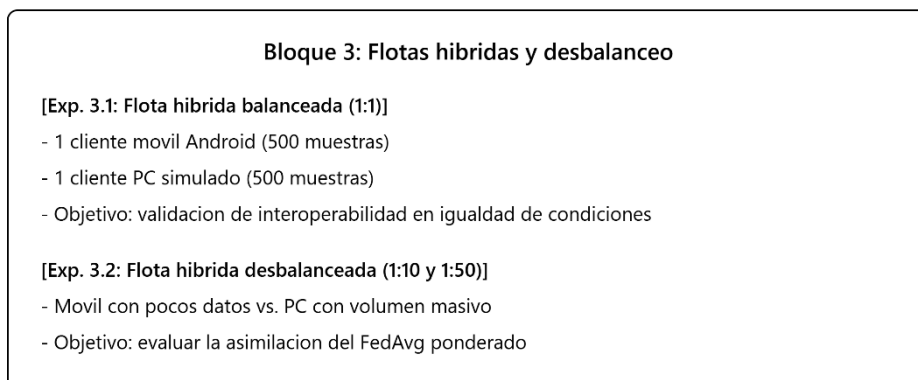


Figura 5.4.1: Experimentos Bloque 3

- Experimento 3.1 (Flota híbrida balanceada 1:1): Validación de la interoperabilidad entre un cliente móvil Android y un cliente PC operando en igualdad de condiciones (500 muestras cada uno).
- Experimento 3.2 (Flotas híbridas desbalanceadas 1:10 y 1:50): Evaluación de la asimilación del algoritmo FedAvg ponderado al enfrentar un móvil con escasez de datos frente a un PC con volumen masivo.

En un despliegue comercial estándar, la red estaría formada exclusivamente por teléfonos móviles. La inclusión de ordenadores en esta fase de pruebas de laboratorio es interesante ya que cumple propósitos técnicos específicos.

- Interoperabilidad garantizada: Al hacer que un móvil (ARM) y un PC (x86) colaboren sin errores numéricos, demostramos que nuestro protocolo de comunicación funciona perfectamente sin importar el hardware subyacente.
- Aislamiento algorítmico: Forzar a un teléfono a entrenar miles de imágenes de golpe provocaría sobrecalentamiento y bloqueos por temperatura (*thermal throttling*). Delegar esta carga pesada al PC nos permite estudiar el comportamiento matemático puro del algoritmo FedAvg sin que los fallos físicos del móvil falseen los resultados.
- Simulación de ecosistemas reales: El ordenador actúa como si fuera un servidor local o un nodo de apoyo (Edge Server). Esto prueba que la plataforma es flexible y capaz de hacer convivir dispositivos potentes junto a teléfonos con recursos limitados dentro de la misma red.

5.4.1. Experimento 3.1: Flota Híbrida Equilibrada (Proporción 1:1)

- **Objetivo:** Verificar la estabilidad de la agregación ponderada y la interoperabilidad en tiempo real cuando un terminal físico Android y un cliente Python en PC participan simultáneamente con cuotas de datos idénticas.
- **Diseño del Escenario:** Se configura una red mixta de dos nodos donde el móvil físico y el agente simulado disponen de exactamente 100 muestras de entrenamiento locales cada uno ($n_{\text{móvil}} = n_{\text{PC}} = 100$). Ambos operan con los mismos hiperparámetros de optimización durante 10 rondas.

5.4.2. Experimento 3.2: Flotas Híbridas Desbalanceadas (Proporciones 1:10 y 1:50)

- **Objetivo:** Evaluar el comportamiento del algoritmo FedAvg cuando existe una disparidad severa en la cantidad de datos aportados por los nodos periféricos, simulando un escenario donde un dispositivo móvil con recursos de almacenamiento limitados interactúa con servidores o estaciones de trabajo.
- **Diseño del Escenario:** Se evalúan dos casos de asimetría estadística:
 - Desbalanceo Moderado (1:10): El cliente Android procesa $n_{\text{móvil}} = 100$ muestras, mientras que el cliente PC entrena sobre $n_{\text{PC}} = 1.000$ muestras.
 - Desbalanceo Severo (1:50): El cliente Android procesa $n_{\text{móvil}} = 100$ muestras frente a las $n_{\text{PC}} = 5.000$ muestras asignadas al nodo simulado.
- **Hipótesis:** El ensayo analizará si esta asimetría suprime la influencia del nodo móvil sobre el modelo global o si, por el contrario, permite incorporar gradientes útiles preservando la estabilidad del sistema.

Capítulo 6

Análisis de los Resultados

Contenido

6.1.	Análisis del Bloque 1: Hardware Real y Cuellos de Botella Físicos	55
6.1.1.	Evaluación de la Convergencia Real On-Device (Exp. 1.1).....	55
6.1.2.	Demostración Empírica y Cuantificación del Efecto <i>Straggler</i> (Exp. 1.2)	57
6.2.	Análisis del Bloque 2: Simulaciones a Escala y Eficiencia de Red	59
6.2.1.	Divergencia de Pesos en Entornos IID vs. Non-IID (Exp. 2.1)....	60
6.2.2.	Rentabilidad Temporal y Ahorro de Ancho de Banda (Exp. 2.2).	65
6.2.3.	Comparativa de Compresión y Cuantización (Exp. 2.3)	68
6.2.4.	Escalabilidad del Tráfico y Degradación Asintótica (Exp. 2.4)	72
6.3.	Análisis del Bloque 3: Flotas Mixtas y Agregación Desbalanceada	76
6.3.1.	Convergencia en Escenarios Equilibrados 1:1 (Exp. 3.1)	76
6.3.2.	Simbiosis y Asimilación del FedAvg Ponderado 1:10 y 1:50 (Exp. 3.2)	78

6.1. Análisis del Bloque 1: Hardware Real y Cuellos de Botella Físicos

El objetivo primordial de esta primera fase experimental consistió en corroborar el funcionamiento del motor de *Machine Learning* integrado en los dispositivos móviles Android y medir el impacto que ejerce la disparidad de procesadores sobre la latencia de la red.

6.1.1. Evaluación de la Convergencia Real On-Device (Exp. 1.1)

El primer ensayo aísla un único dispositivo móvil operando como cliente exclusivo para validar el cálculo de gradientes en procesadores de arquitectura ARM. Los registros del sistema verifican que el intérprete LiteRT es capaz de ejecutar la retropropagación *on-device* de forma estable.

Tal y como se observa en las métricas extraídas en las figuras 6.1.1 y 6.1.2, la precisión predictiva (*Accuracy*) asciende progresivamente desde un 12.78% inicial en la primera ronda hasta alcanzar un 45.47% en la ronda 50.

La idea aquí no es conseguir un modelo superpreciso, si no ver que funciona realmente de manera correcta.

```
Recibido Cliente Android_1 | Muestras: 100 | Ronda: 1

|Ejecutando FedAvg Ponderado! (Ronda 1)...

--- RESULTADOS RONDA 1 ---

Accuracy: 12.78%

Loss: 2.2931

Tráfico: 795.08 KB

Resultados de la ronda 1 guardados en resultados_experimentos.csv
```

Figura 6.1.1: Captura Terminal Ronda 1 Experimento 1.1

```
Recibido Cliente Android_1 | Muestras: 100 | Ronda: 50

|Ejecutando FedAvg Ponderado! (Ronda 50)...

--- RESULTADOS RONDA 50 ---

Accuracy: 45.47%

Loss: 1.8977

Tráfico: 795.08 KB

Resultados de la ronda 50 guardados en resultados_experimentos.csv

>> OBJETIVO ALCANZADO (90% Acc) o RONDAS FINALIZADAS, Deteniendo experimento... <<<
```

Figura 6.1.2: Captura Terminal Ronda 50 Experimento 1.1

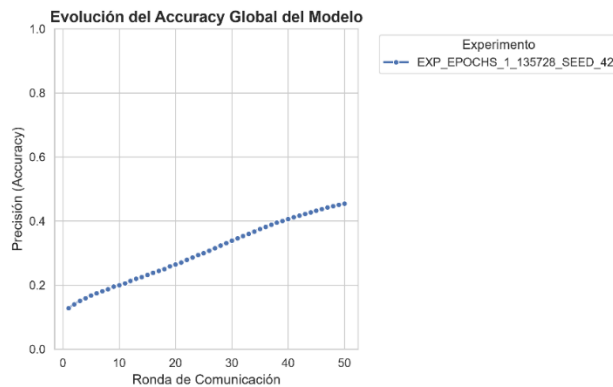


Figura 6.1.3: Gráfica Accuracy Experimento 1.1

Al operar de forma aislada, el parámetro correspondiente a la espera de rezagados (`Espera_Rezagados_Seg`) arroja un valor nulo (0.0 s), esto se aprecia en el archivo de resultados .csv que se crea cuando finaliza el entrenamiento. Esto permite establecer la latencia base telemática del dispositivo, con un tiempo total medio por ronda de aproximadamente 5.85 s, de los cuales el entrenamiento local sobre las imágenes requiere 0.5 s. Esto lo podemos ver en la figura 6.1.4.

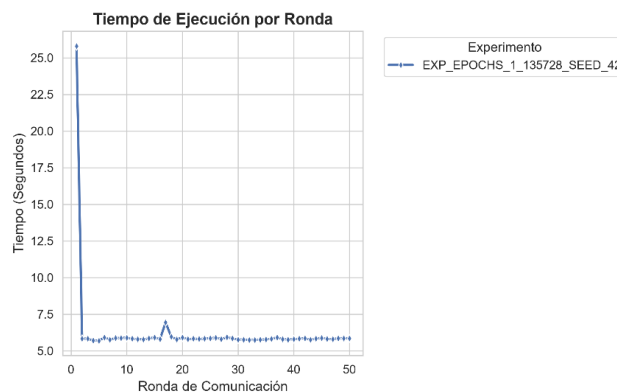


Figura 6.1.4: Gráfica Tiempo Ejecución por Ronda Experimento 1.1

El pico inicial que se aprecia en la figura 6.1.4 es un comportamiento clásico y normal conocido como Arranque en Frío. En la primera iteración el sistema ejecuta una serie de tareas pesadas de configuración que no se vuelven a repetir en todo el experimento, por eso una vez el sistema supera esta fase de arranque en la ronda 1, su latencia real se desploma y se estabiliza.

Este comportamiento valida la robustez arquitectónica de la aplicación nativa, certificando su capacidad para computar y serializar matrices de pesos de forma continua sin incurrir en desbordamientos de memoria (*Out Of Memory*).

6.1.2. Demostración Empírica y Cuantificación del Efecto *Straggler* (Exp. 1.2)

En este ensayo se configura una red mixta compuesta por un terminal de alta capacidad (Samsung Galaxy S25) y dos terminales de capacidad estándar (Samsung Galaxy M23).

En la gráfica 6.1.5 observamos que, la precisión inicial registrada en la Ronda 1 fue de $11.63\% \pm 3.42\%$, alcanzando una precisión final de $47.07\% \pm 1.95\%$ hacia la Ronda 30, lo que evidencia una mejora sostenida a lo largo del entrenamiento.

En cuanto al Tiempo Medio de Entrenamiento Local, se obtuvo un valor de $1.02s \pm 0.01s$. Este dato representa la media aritmética entre el tiempo del dispositivo S25 y el tiempo algo superior registrado por los dos dispositivos M23.

Respecto al Tiempo Total de Ronda (Tiempo_Ronda_Seg), como vemos en la Figura 6.1.7, se registró un valor de $6.63s \pm 1.58s$, aunque se observa un comportamiento muy estable de $6.32s \pm 0.06$ si se excluyen los picos iniciales de conexión en frío detectados en las rondas 7 y 13 de algunas semillas, particularmente en la semilla 456.

Finalmente, el Tiempo Perdido por Rezagados (Espera_Rezagados_Seg) fue de $4.42s \pm 1.21s$ ya que al inicio el terminal de gama alta (Galaxy S25) también invierte tiempo extra en inicializar y cargar datos en memoria, lo que reduce temporalmente su ventaja frente a los otros dispositivos más lentos (Galaxy M23). Sin embargo, una vez estabilizado el sistema al finalizar el entrenamiento tan rápido, el tiempo que pasa inactivo esperando a que el hardware más limitado termine de procesar su lote se maximiza, por lo que este valor de espera se estabiliza de forma contundente aumentando aproximadamente a 4.98 segundos, revelando la brecha de rendimiento entre gamas y demostrando que la red siempre quedará sujeta a la velocidad del dispositivo más rezagado.

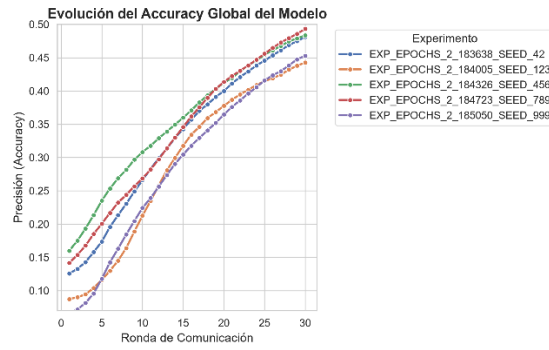


Figura 6.1.5: Accuracy Experimento 1.2

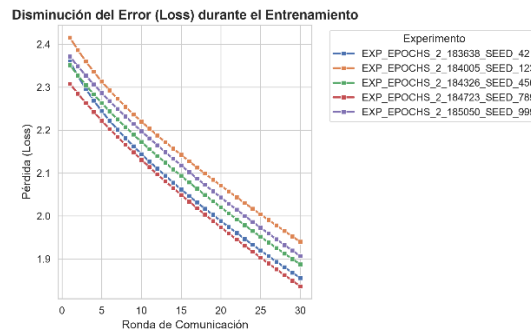


Figura 6.1.6: Loss Experimento 1.2

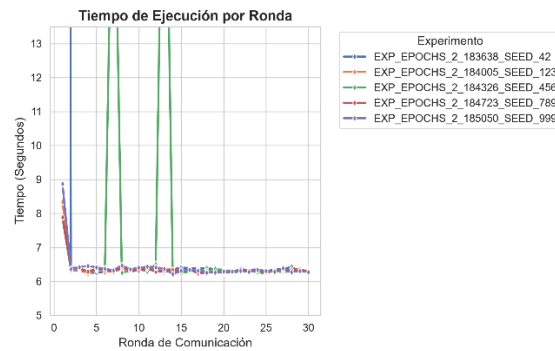


Figura 6.1.7: Tiempo Ejecución por Ronda Experimento 1.2

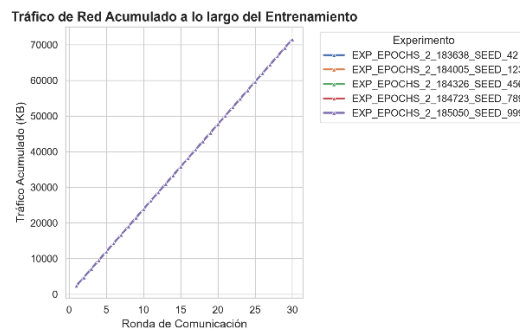


Figura 6.1.8: Tráfico Acumulado Experimento 1.2

La figura 6.1.8 ilustra un crecimiento perfectamente lineal del tráfico de red acumulado a lo largo de las rondas de comunicación.

La superposición exacta de todas las ejecuciones (independientemente de la semilla estadística) demuestra que el consumo de ancho de banda es constante en cada iteración. Esto confirma que el volumen de datos transmitidos depende exclusivamente del tamaño de la arquitectura de la red neuronal, manteniéndose inalterable frente a la variabilidad de los datos o el progreso local del entrenamiento.

A. Demostración Empírica del Cuello de Botella por Rezagados (*Straggler Effect*)

A lo largo de las 30 rondas de experimentación, el servidor experimentó un tiempo medio de Espera de Rezagados de $4.42s \pm 1.21s$ por ronda, estabilizándose en 4.98 s en condiciones de régimen permanente. Este valor demuestra que un dispositivo de gama alta (Samsung Galaxy S25) finaliza sus operaciones aritméticas y permanece esperando al último dispositivo de gama media (Samsung Galaxy M23) para poder agregar el modelo. Por tanto, la velocidad teórica de convergencia de una red federada del mundo real está acotada superiormente por el nodo con menor capacidad computacional.

B. Impacto Relativo de la Ineficiencia Temporal

Al relacionar los tiempos locales y globales, se observa que la latencia de espera del rezagado (4.98 s) representa aproximadamente el 78.8% del tiempo total de la ronda (6.32 s). El cálculo algorítmico efectivo representa solo una fracción del ciclo, desperdiándose más de tres cuartas partes del tiempo total en sincronización y latencia de red.

6.2. Análisis del Bloque 2: Simulaciones a Escala y Eficiencia de Red

Mediante la paralelización de clientes simulados, este bloque suprime las limitaciones físicas, derivadas por ejemplo del sobrecalentamiento de los dispositivos móviles, para así analizar rigurosamente el comportamiento de los

paquetes de datos y la convergencia matemática a gran escala bajo condiciones de red estresadas, sin que las caídas de rendimiento del hardware físico enmascaren los resultados de la red.

6.2.1. Divergencia de Pesos en Entornos IID vs. Non-IID (Exp. 2.1)

- **Caso A (Entorno IID):** Datos balanceados. Cada cliente tiene 1.000 imágenes mezcladas de todos los números (0 al 9).

En las gráficas 6.2.1 y 6.2.2 podemos ver los siguientes parámetros de *Accuracy* y *Loss*:

En la Ronda 1, correspondiente al punto de partida del entrenamiento, se registró un rendimiento medio de *Accuracy* = 84.934% y *Loss* = 0.62024.

El hito de convergencia rápida se alcanza de manera estable y consistente entre la Ronda 3 y la Ronda 4 en todas las repeticiones realizadas, con una *Accuracy* media de 90.026% en la Ronda 3 y de 90.764% en la Ronda 4.

Al finalizar el proceso, en la Ronda 30, se observa una convergencia final con una *Accuracy* de 94.216% y una *Loss* de 0.19836.

En cuanto al tráfico de red, se registró un valor acumulado de 239.10MB (239103.52KB), resultando exactos e idénticos en las 5 repeticiones realizadas, lo que evidencia la consistencia arquitectónica del sistema.

Por último, el tiempo medio por ronda fue de 5.45s (excluyendo la inicialización en frío correspondiente a la primera ronda), mientras que el tiempo medio de entrenamiento local se situó en 0.49s por cliente y ronda.

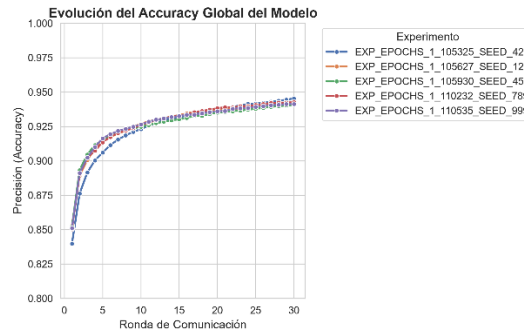


Figura 6.2.1: Accuracy IID Experimento 2.1

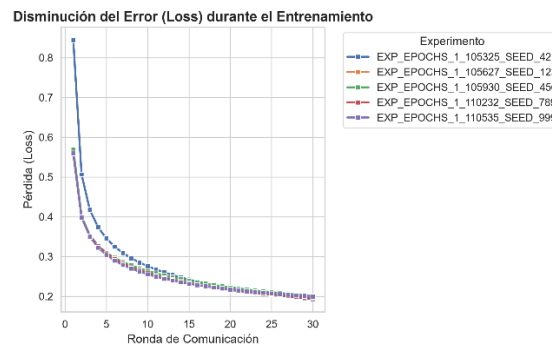


Figura 6.2.2: Loss IID Experimento 2.1

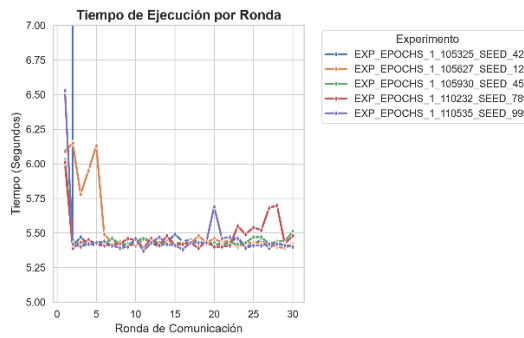


Figura 6.2.3: Tiempo Ejecución por Ronda IID Experimento 2.1

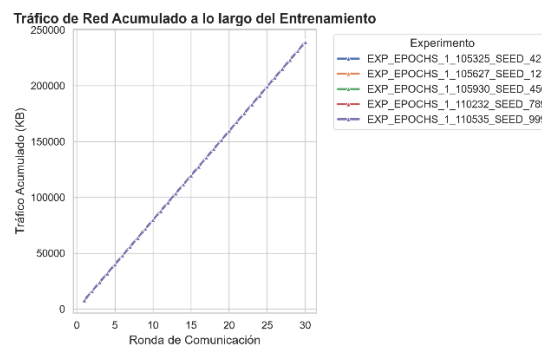


Figura 6.2.4: Tráfico Acumulado IID Experimento 2.1

A. Alta Estabilidad y Baja Dispersión en Distribuciones IID

El análisis estadístico de las 5 repeticiones demuestra que, en un entorno con datos IID, el Aprendizaje Federado exhibe una varianza extremadamente baja de precisión en la ronda final. La homogeneidad en las distribuciones de datos de los 10 clientes garantiza que los gradientes locales calculados en cada epoch apunten hacia un mínimo global coincidente, eliminando oscilaciones o sesgos en el servidor tras aplicar el FedAvg.

B. Convergencia Temprana vs. Coste de Telecomunicación

El modelo supera el umbral objetivo del 90% de precisión en apenas 4 rondas (equivalente a un consumo de solo 31.88MB de tráfico total de red y alrededor de 22seg de tiempo acumulado). Las 26 rondas posteriores aportan una ganancia marginal del 3.44% extra de precisión a cambio de consumir otros 207.22MB. Este dato evidencia la necesidad de implementar mecanismos de parada temprana (*Early Stopping*) en arquitecturas reales para optimizar el consumo energético y el ancho de banda.

C. Relación de Cuellos de Botella (Computación vs. Red)

Los datos registrados confirman que, para redes poco profundas como la evaluada en MNIST, el cuello de botella del sistema radica en la latencia de comunicación y sincronización del servidor, no en el cómputo en el dispositivo. El tiempo local medio de entrenamiento de un cliente representa apenas un 9% del tiempo total de la ronda (0.49 s de cómputo frente a 5.45 s de ciclo de ronda completo), justificando la idoneidad del modelo para dispositivos con procesadores limitados.

- **Caso B (Entorno Non-IID):** Partición estricta por etiquetas. El Cliente 1 solo verá imágenes del dígito 0 y 1; el Cliente 2 solo verá 2 y 3, el Cliente 3 solo 4 y 5, etc.

En las figuras 6.2.5 y 6.2.6 podemos ver el comportamiento del *Accuracy* y del *Loss* a lo largo del entrenamiento.

En la Ronda 1, correspondiente al punto de partida del entrenamiento, se registró un rendimiento medio de *Accuracy* = 44.574% y *Loss* = 1.7729.

Durante el tramo intermedio del entrenamiento, comprendido entre la Ronda 15 y la Ronda 30, el modelo presenta oscilaciones severas, manifestadas en forma de caídas y subidas abruptas con un patrón de "diente de sierra". A modo de ejemplo, en la semilla 42 se observa cómo la Accuracy pasa de un 60.5% en la Ronda 24 a un 57.1% en la Ronda 25, para posteriormente rebotar hasta 60.5% en la Ronda 26. Este comportamiento se aprecia con mayor claridad en las gráficas mostradas a continuación.

Al finalizar el proceso, en la Ronda 30, se observa una convergencia final con una Accuracy de 64.926% y una Loss de 1.06996.

En cuanto al tráfico de red, se registró un valor acumulado de 239.10MB (239103.52KB), resultando exactos e idénticos en las 5 repeticiones realizadas, lo que evidencia la consistencia arquitectónica del sistema.

Por último, el tiempo medio de entrenamiento local se situó en 0.38s por cliente y ronda.

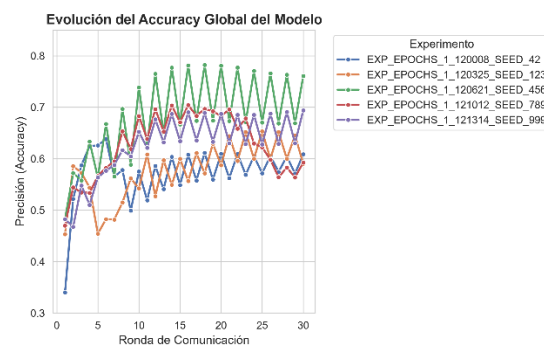


Figura 6.2.5: Accuracy Non-IID Experimento 2.1

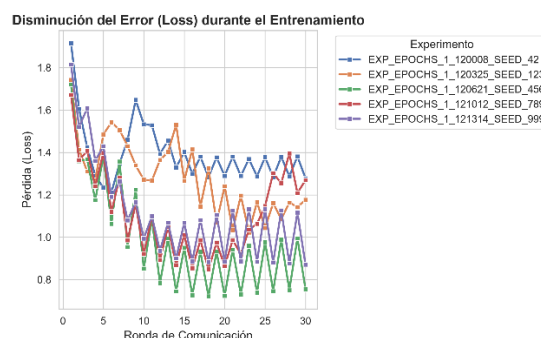


Figura 6.2.6: Loss Non-IID Experimento 2.1

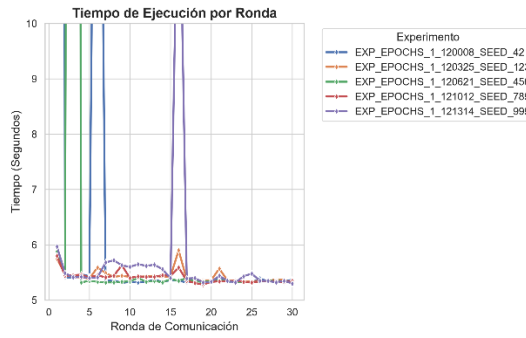


Figura 6.2.7: Tiempo Ejecución por Ronda Non-IID Experimento 2.1

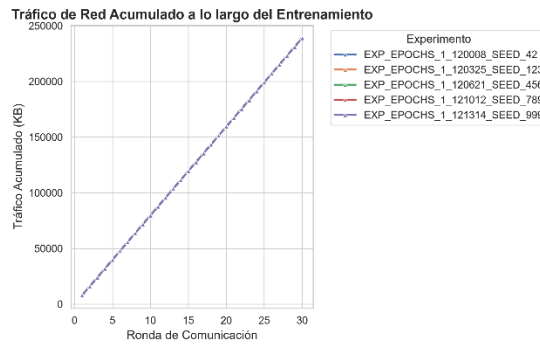


Figura 6.2.8: Tráfico Acumulado Non-IID Experimento 2.1

En la figura 6.2.7 podemos observar que existen unos picos en el tiempo de ejecución por ronda, esto es debido a interferencias y bloqueos puntuales del sistema operativo del PC. Como el servidor tiene que esperar obligatoriamente a que todos los participantes terminen, si el ordenador sufre un pequeño bloqueo y retrasa a un solo cliente simulado, toda la ronda se frena. Por tanto, esos picos de tiempo extra reflejan un atasco puntual del PC, no un problema matemático del algoritmo.

A. Demostración Empírica de la Divergencia de Pesos (Weight Divergence)

La comparativa entre los escenarios IID y Non-IID confirma empíricamente la vulnerabilidad del algoritmo FedAvg ante datos patológicamente sesgados. Mientras que en IID el modelo roza el 90% de precisión en la Ronda 3, en Non-IID apenas alcanza una precisión media del 64.92% tras 30 rondas y el mismo gasto de ancho de banda (239.10 MB). Al entrenar cada cliente exclusivamente con 2 clases (ej. Cliente 1 con '0' y '1', Cliente 2 con '2' y '3'), los gradientes

locales apuntan hacia óptimos locales incompatibles entre sí, generando una penalización de -29.29% aprox. de Accuracy global.

B. Inestabilidad y Alta Varianza en la Convergencia

El análisis de dispersión de las 5 repeticiones revela que el sesgo de datos multiplica la incertidumbre del sistema. La curva de aprendizaje presenta un comportamiento altamente oscilatorio ('dientes de sierra'), donde el Accuracy sube y baja entre rondas consecutivas en lugar de ascender monótonamente. Esto demuestra que la heterogeneidad estadística de los datos de usuario introduce turbulencias severas en la convergencia del modelo federado.

C. Independencia entre Coste Computacional y Calidad del Dato

Desde el punto de vista de sistemas, los tiempos registrados confirman que el hardware y la red consumen los mismos recursos independientemente de la utilidad matemática del dato. El tráfico de red (7 970.12 KB/ronda) y el tiempo medio de cómputo local ($\sim 0.38\text{-}0.49$ s) son invariantes respecto a la distribución IID o Non-IID.

6.2.2. Rentabilidad Temporal y Ahorro de Ancho de Banda (Exp. 2.2)

La modulación de las épocas locales de entrenamiento (E) revela un compromiso (trade-off) decisivo entre la computación periférica y el consumo del espectro radioeléctrico. Para este ensayo, se establece como meta alcanzar una precisión global del 92.5%.

- **Con 1 época local:** El sistema requiere una media de 19.2 rondas para alcanzar el objetivo, consumiendo 76.513 MB de red, con un tiempo de cómputo local de $0.30\text{ s} \pm 0.08\text{ s}$.
- **Con 5 épocas locales:** Se alcanza el objetivo en 5.2 rondas, requiriendo 20.722 MB de red, con un tiempo de cómputo local de $0.93\text{ s} \pm 0.18\text{ s}$.
- **Con 10 épocas locales:** Se alcanza el objetivo en 3 rondas medias con una media del 92.694% de accuracy requiriendo solo 11.955 MB, elevando el tiempo local a $1.46\text{ s} \pm 0.44\text{ s}$.

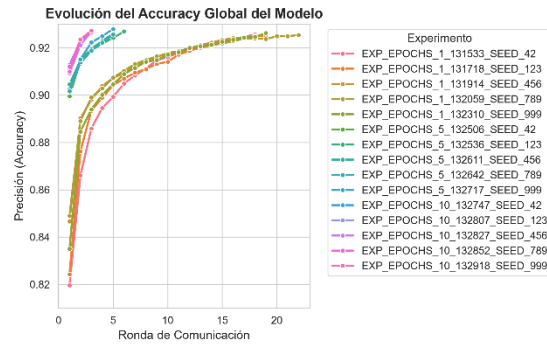


Figura 6.2.9: Accuracy Experimento 2.2

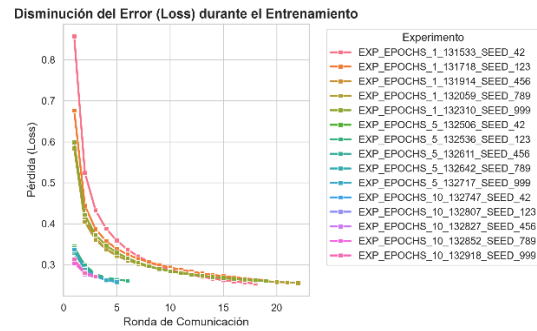


Figura 6.2.10: Loss Experimento 2.2

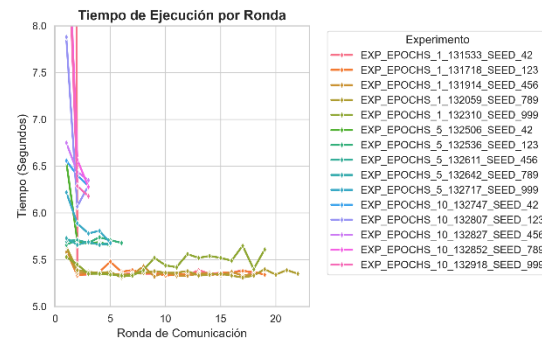


Figura 6.2.11: Tiempo Ejecución por Ronda Experimento 2.2

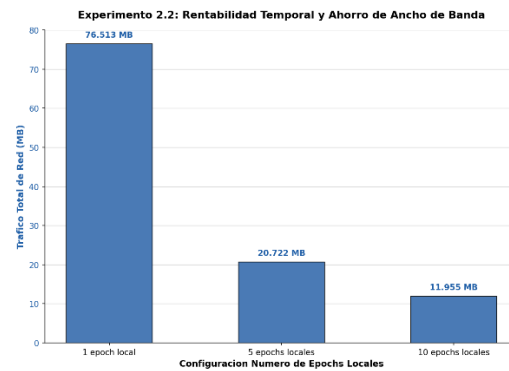


Figura 6.2.12: Tráfico Acumulado Experimento 2.2

A. Ahorro Masivo de Ancho de Banda al Subir el Umbral de Exigencia

Al fijar un objetivo de precisión exigente del 92.5%, se evidencia la extrema ineficiencia en comunicaciones de utilizar una única época local. Una configuración de 1 época local requiere 19 rondas de agregación y transmite 76.51 MB por la red para alcanzar la convergencia. Al incrementar el entrenamiento local a 5 épocas locales, el sistema alcanza el mismo objetivo en solo 5 rondas, reduciendo el tráfico global un 72.56% (20.72 MB). Si se eleva a 10 épocas, se requieren 3 rondas (11.955 MB), lo que supone un ahorro total de 84.16% de datos transmitidos frente a la línea base.

B. El Trade-Off Físico: Cuándo Detener las Épocas Locales

Para calcular el tiempo total de uso del dispositivo, es vital sumar el coste telemático (conexión y transmisión de pesos) y el coste de cómputo (procesamiento matemático local). Bajo esta premisa, el escenario de 19 rondas con 1 época es el más ineficiente: obliga al teléfono a resolver latencias de red y transmitir megabytes de datos 19 veces distintas. Esto consume significativamente más tiempo total y batería que asumir una mayor carga matemática a cambio de realizar tan solo 3 conexiones ($E=10$).

Sin embargo, llevar el cómputo local al extremo tampoco es la solución perfecta. Al exigir un 92.5% de precisión, el paso de 5 a 10 épocas sí aporta una mejora al ahorrar 2 rondas de red (3 rondas frente a 5 rondas). Aunque este ahorro en telecomunicaciones minimiza el tráfico a 11.96 MB (lo cual está plenamente justificado en escenarios donde el ancho de banda sea extremadamente crítico o costoso, como redes 3G/4G congestionadas), penaliza de forma directa la computación en el dispositivo: el tiempo de entrenamiento por ronda crece un 57% (de 0.93 s a 1.46 s).

Por tanto, el balance definitivo de la arquitectura se establece de la siguiente manera:

- 1 época local: Inviabile por el excesivo coste de tiempo y energía que supone la transmisión continua por red (19 rondas).
- 5 épocas locales: Representa el punto de equilibrio óptimo. Reduce drásticamente el cuello de botella telemático frente a la época única, pero

evita el sobreesfuerzo de CPU y batería que sufren las flotas móviles al intentar procesar 10 épocas.

- 10 épocas locales: Ideal exclusivamente para entornos de conectividad muy limitada, asumiendo un alto consumo de los recursos del terminal.

6.2.3. Comparativa de Compresión y Cuantización (Exp. 2.3)

En este experimento se presenta que la evaluación de técnicas de telecomunicación aplicadas a la serialización de parámetros demuestra la superioridad de la reducción aritmética frente a la compresión tradicional.

A continuación, se muestran las gráficas de las métricas obtenidas para cada caso de la experimentación.

▪ Gráfica Accuracy

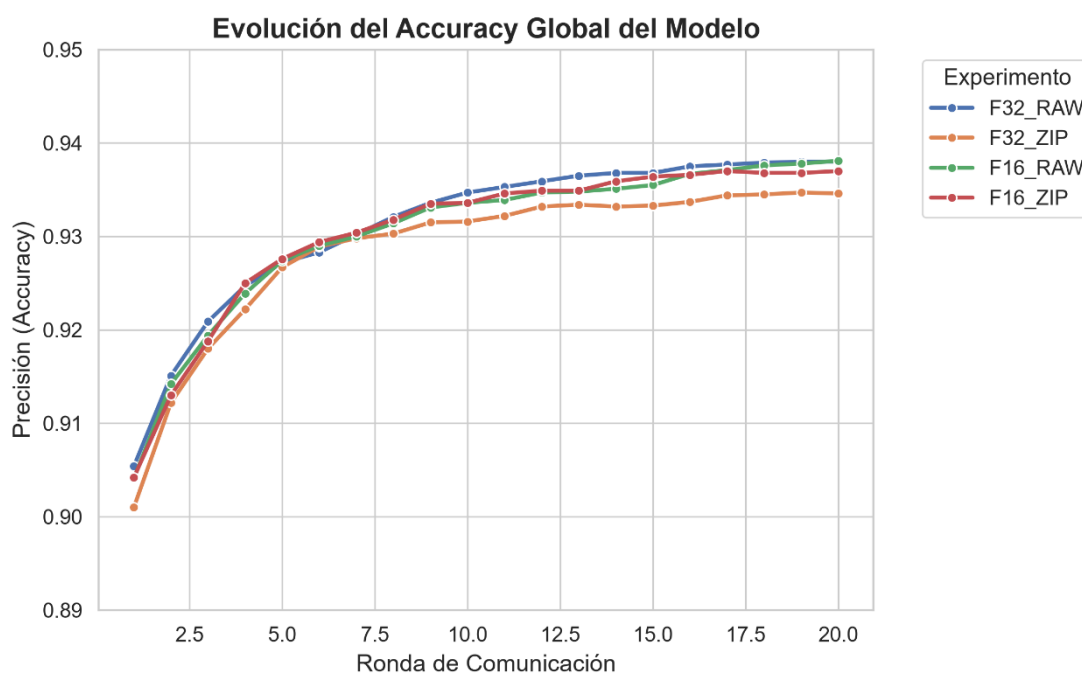


Figura 6.2.13: Accuracy Experimento casos 2.3

▪ Gráfica Loss

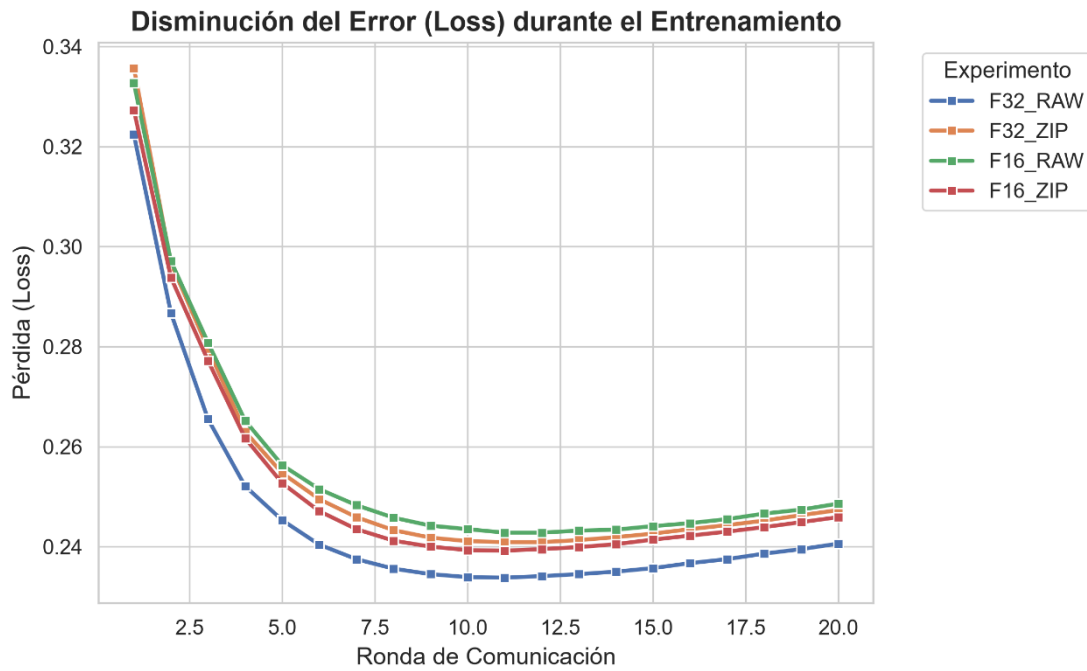


Figura 6.2.14: Loss Experimento casos 2.3

▪ Gráfica Tiempo de Ejecución por Ronda

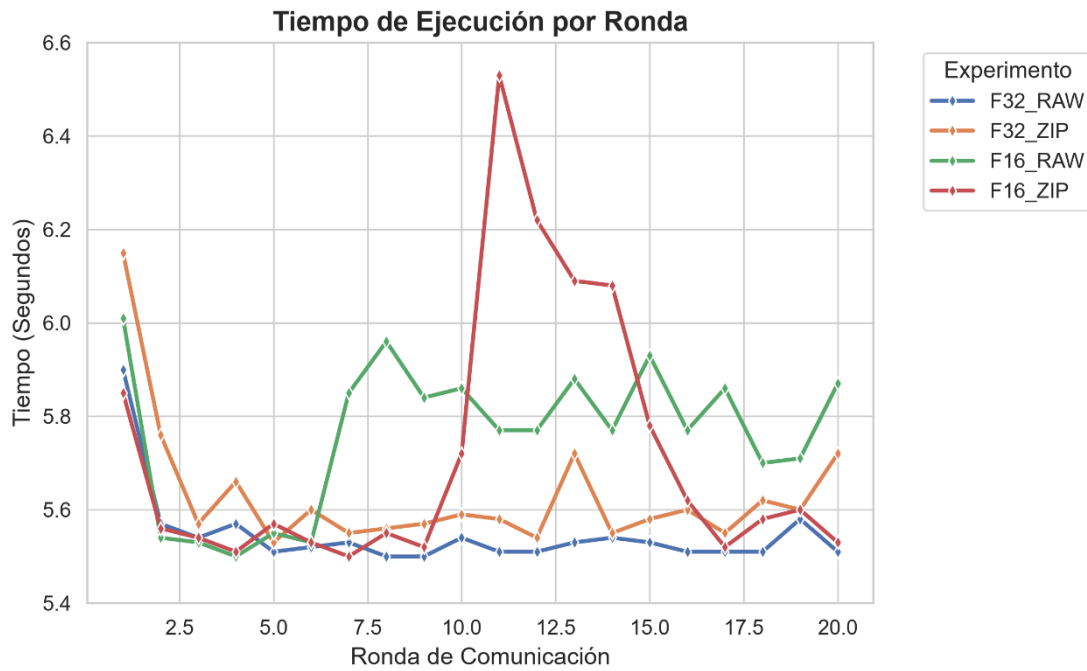


Figura 6.2.15: Tiempo Ejecución por Ronda Experimento casos 2.3

▪ **Gráfica Tráfico Acumulado**

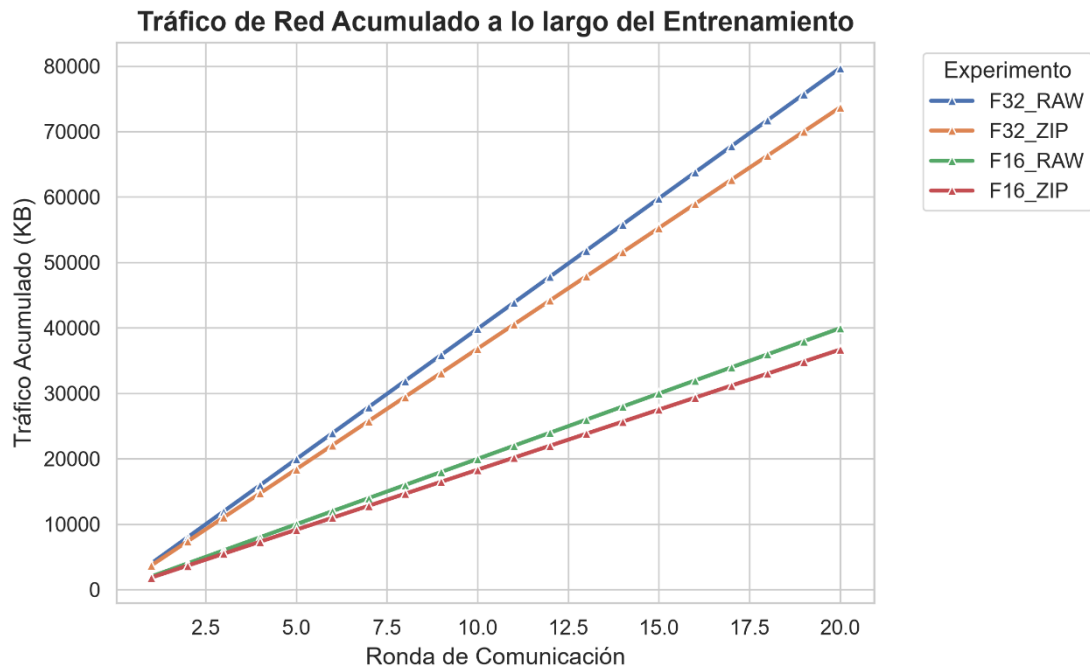


Figura 6.2.16: Tráfico Acumulado Experimento casos 2.3

En la siguiente Tabla se resumen las métricas telemáticas correspondientes al envío de los parámetros sinápticos durante 20 rondas de comunicación:

Modo	Precisión Datos	Compresión	Tráfico por Ronda	Ahorro Ancho de Banda	Accuracy Final	Tiempo Local
F32_RAW	Float32 (4bytes)	No	3985.06 KB	0.0% (Referencia)	93.83%	0.55 s
F32_ZIP	Float32 (4bytes)	Si	3684.22 KB	5.35%	93.65%	0.73 s
F16_RAW	Float16 (2bytes)	No	1997.36 KB	49.88%	93.66%	0.77 s
F16_ZIP	Float16 (2bytes)	Si	1834.49 KB	54.23%	93.65%	0.73 s

Tabla 1: Métricas Experimento 2.3

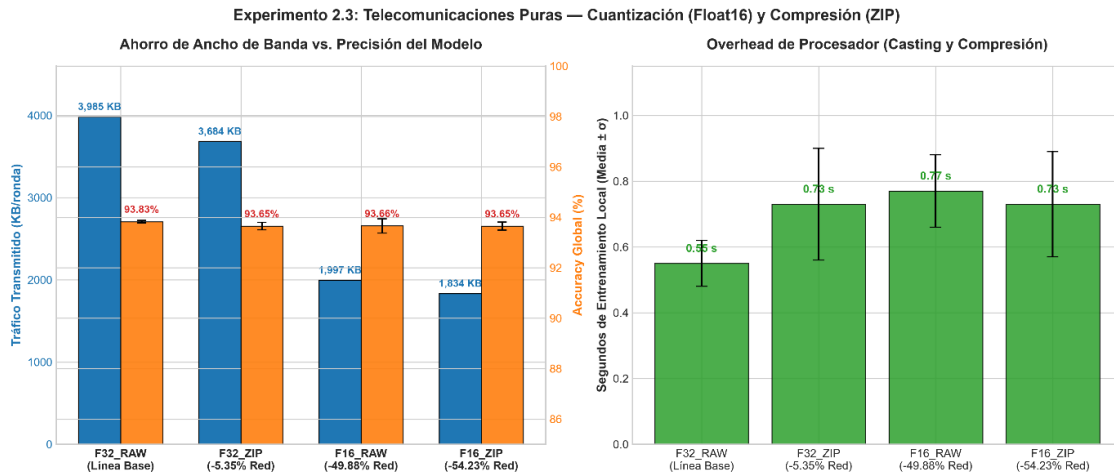


Figura 6.2.17: Gráfica Comparativa Cuantización y Compresión Experimento 2.3

A. Eficiencia Superior de la Cuantización frente a la Compresión Genérica:

Los resultados empíricos prueban que reducir la precisión aritmética de 32 a 16 bits (F16_RAW) genera un ahorro de red prácticamente lineal (49.88%), superando al algoritmo de compresión genérico en 32 bits (F32_ZIP), el cual solo alcanza un 5.35% de reducción.

B. Sinergia Multicapa (F16_ZIP):

Al combinar la cuantización con compresión sin pérdidas (F16_ZIP), el tráfico por ronda se reduce en un - 54.23% (1834.49 KB frente a los 3985.06 KB de la línea base). Esta optimización estructural reduce la factura de datos sin penalizar la convergencia, pues el Accuracy global final se mantiene en un 93.65% (una pérdida de rendimiento marginal de tan solo el - 0.18%).

C. El Compromiso Procesador-Red (Overhead en el Dispositivo):

El ahorro en telecomunicaciones conlleva un pequeño coste computacional en el dispositivo cliente. Las operaciones de conversión de tipos en memoria y compresión elevan el tiempo local medio en +180 ms por ronda (0.73 s en F16_ZIP frente a 0.55 s en F32_RAW). En un entorno móvil real, este coste de CPU es intrascendente en comparación con el ahorro energético y temporal que supone transmitir la mitad de los megabytes por interfaz radioeléctrica 4G/5G. Esto es teniendo en cuenta solo el tiempo, pero se podrían considerar otros

factores como la sobrecarga en el consumo energético del dispositivo, aunque este no está considerado en este estudio.

6.2.4. Escalabilidad del Tráfico y Degradación Asintótica (Exp. 2.4)

En esta parte se demuestra cómo el incremento de clientes conectados manteniendo un volumen global de datos constante (10.000 muestras) verifica el modelo de crecimiento del sistema federado.

A continuación, tenemos una serie de gráficas que muestran las medidas obtenidas en esta prueba.

▪ Gráfica Accuracy

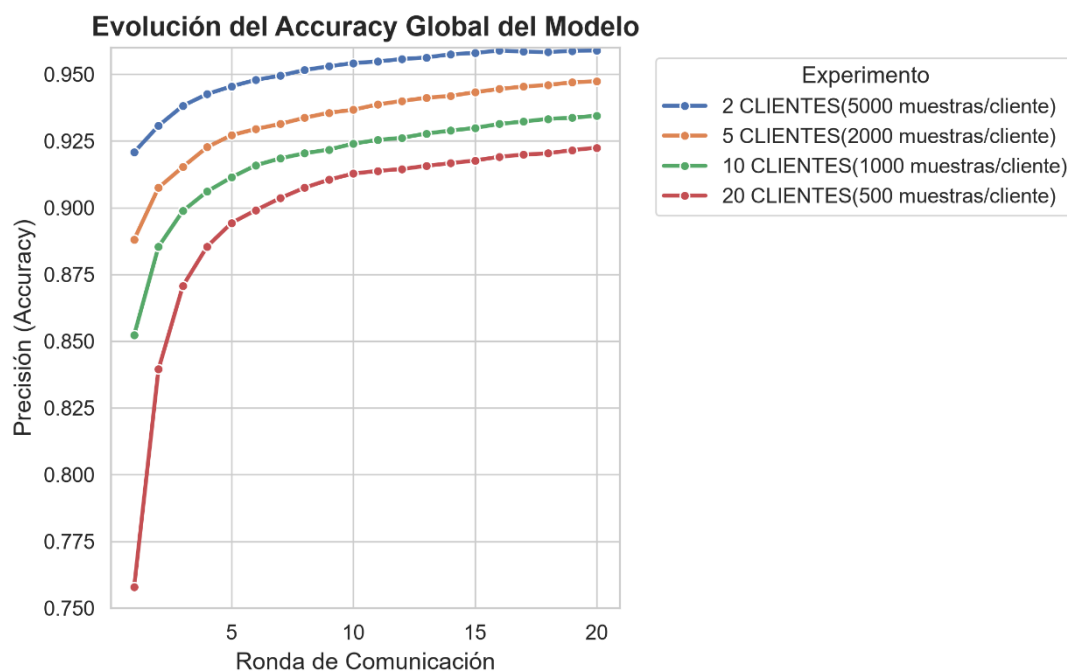


Figura 6.2.18: Accuracy Experimentos 2.4

▪ Gráfica Loss

Disminución del Error (Loss) durante el Entrenamiento

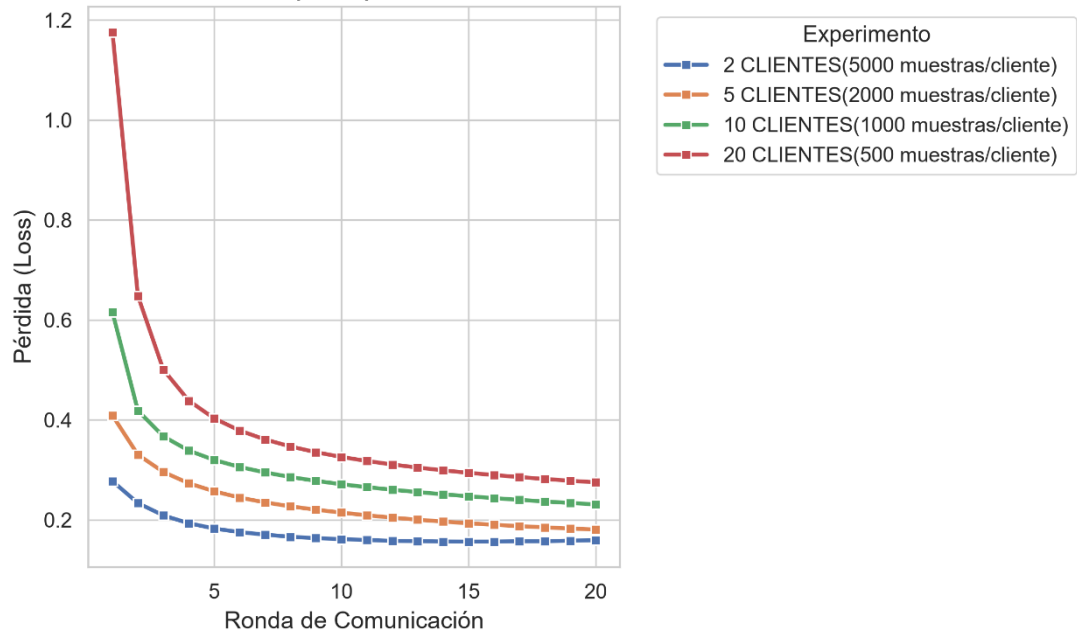


Figura 6.2.19: Loss Experimentos 2.4

▪ Gráfica Tiempo de Ejecución por Ronda

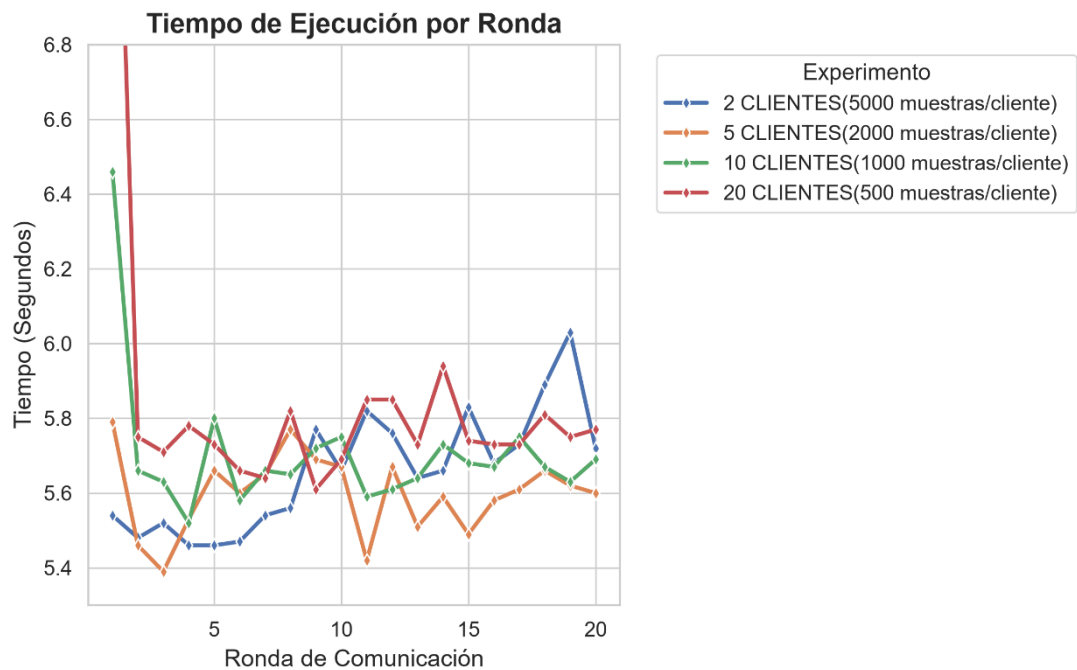


Figura 6.2.20: Tiempo Ejecución por Ronda Experimentos 2.4

▪ **Gráfica Tráfico Acumulado**

Tráfico de Red Acumulado a lo largo del Entrenamiento

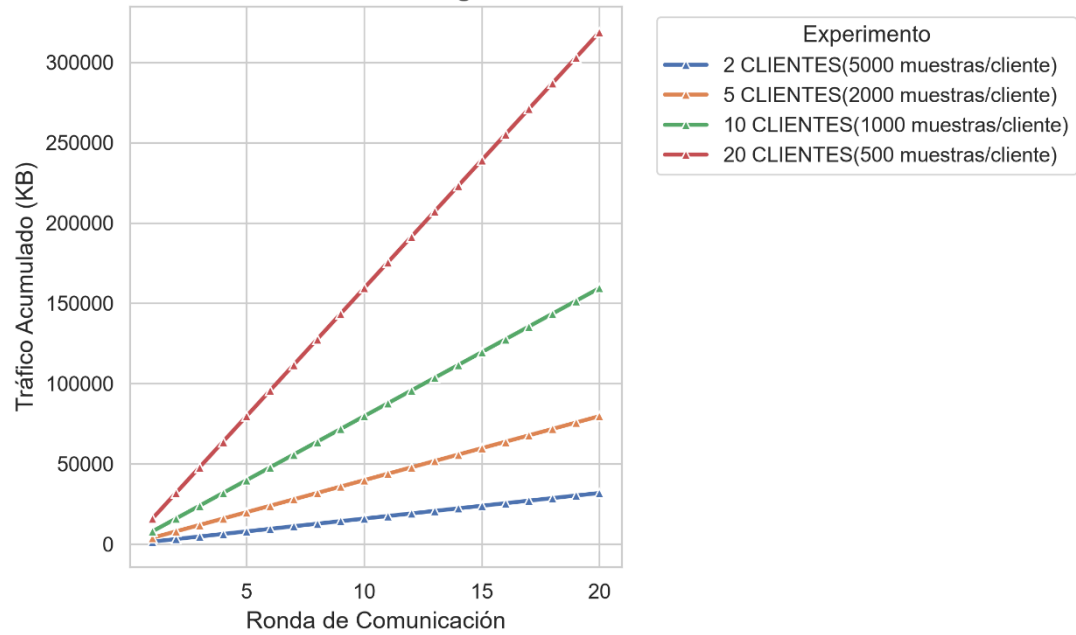


Figura 6.2.21: Tráfico Acumulado Experimentos 2.4

La telemetría extraída de las 4 casuísticas se recoge en la siguiente tabla:

Número Clientes Simulados	Muestras locales/cliente	Tráfico por Ronda	Factor escalado (Tráfico)	Accuracy Final	Tiempo de Ronda
2	5000	1594.02KB	1.0x (Ref)	95.61%	5.58 s
5	2000	3985.06KB	2.5x	94.60%	5.53 s
10	1000	7970.12KB	5.0x	93.46%	5.58 s
20	500	15940.23KB	10.0x	92.36%	5.62 s

Tabla 2: Métricas Experimento 2.4

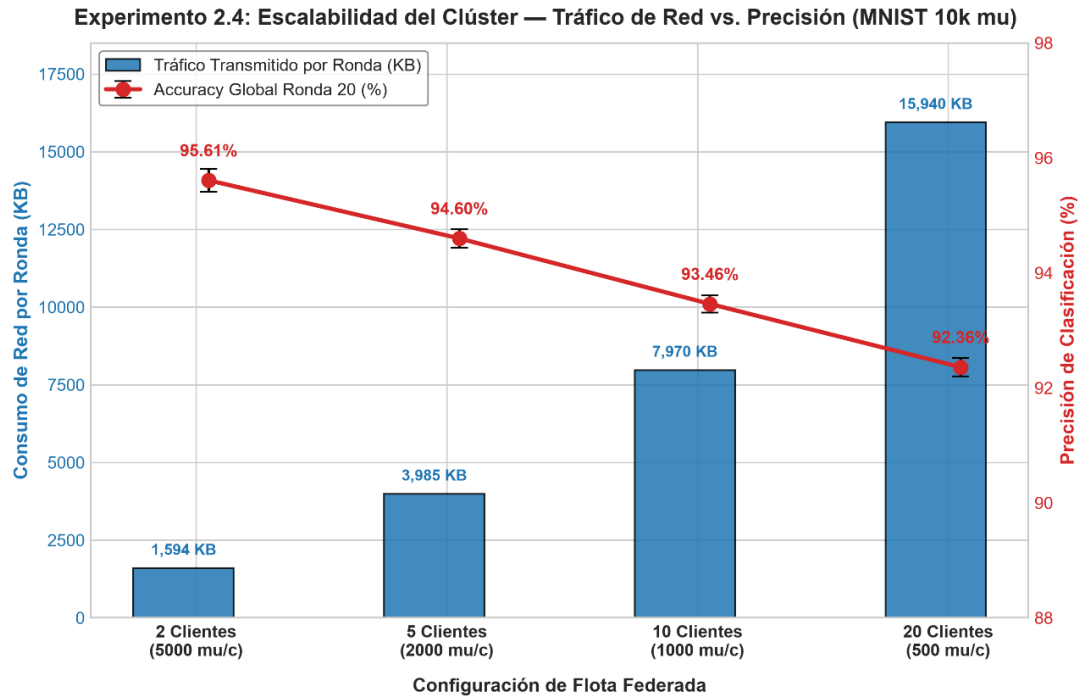


Figura 6.2.22: Gráfica Tráfico y Precisión con Escalabilidad del Clúster Experimento 2.4

A. Independencia de Carga de Datos vs. Carga de Telecomunicación

La telemetría extraída de las 4 casuísticas demuestra que el costo de comunicación de una red federada basada en FedAvg escala en proporción lineal con el conjunto de clientes (N), resultando invariante respecto al volumen global de datos en entrenamiento ($D_{\text{total}} = 10000$). Aunque una flota de 20 dispositivos entrena lotes 10 veces más reducidos que una de 2 dispositivos, el servidor requiere la recepción e inyección de 20 vectores de pesos sinápticos por ronda, disparando el consumo medio de 1.59 MB/ronda a 15.94 MB/ronda.

B. Degradación Asintótica por Granularidad Estocástica

Al repartir los datos en trozos cada vez más pequeños (pasando de 5.000 a solo 500 muestras por cliente), la precisión final en la ronda 20 bajó un poco: de un 95.61% con 2 clientes a un 92.36% con 20.

La pérdida de ese 3.25% se debe a que al entrenar con tan pocos datos, el aprendizaje de cada cliente se vuelve más inestable. Como cada uno tira un poco hacia su lado, al servidor le cuesta más ponerlos de acuerdo, por lo que necesita unas cuantas rondas más para llegar a la misma precisión de antes.

6.3. Análisis del Bloque 3: Flotas Mixtas y Agregación Desbalanceada

Este último bloque evalúa la interoperabilidad real y la resiliencia algorítmica cuando dispositivos Android de bajos recursos cooperan simultáneamente con agentes de alto rendimiento.

En cuanto a la asignación del peso relativo de cada cliente en cada agregación, el servidor aplica la fórmula de McMahan:

$$\text{Peso Relativo del cliente } k = \frac{n_k}{D_{total}} \times 100$$

- n_k = número de muestras que ha entrenado ese cliente en específico.
- D_{total} = suma de todas las muestras de todos los clientes de la flota en esa ronda.

6.3.1. Convergencia en Escenarios Equilibrados 1:1 (Exp. 3.1)

En este primer caso se experimenta con 1 cliente Android y 4 clientes Python simulados, en el que cada uno tiene 100 muestras locales.

Con solo 500 muestras globales (100 por dispositivo), el modelo opera en escasez de datos: la Accuracy final en la Ronda 20 es $77.93\% \pm 1.08\%$ (máximo de 79.49% en la semilla 789), escalando de forma estable desde un 16% inicial sin divergencias.

El tráfico por ronda es 3979.26 KB (3.98 MB), este valor corresponde a la suma en Float32 de las 5 matrices de pesos transmitidas, con un acumulado en 20 rondas de 79585.16 KB (79.58 MB).

El tiempo de entrenamiento local es $0.42 \text{ s} \pm 0.05 \text{ s}$, prácticamente instantáneo en todos los clientes. Mientras que el tiempo total por ronda se estabiliza en $5.97 \text{ s} \pm 0.45 \text{ s}$.

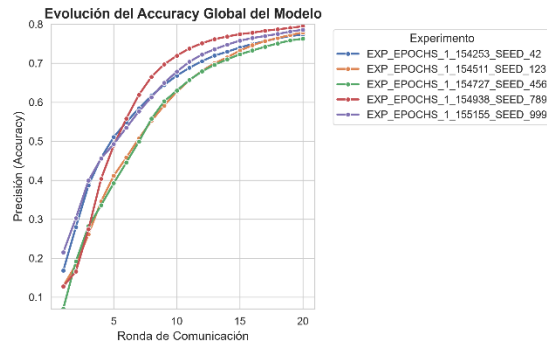


Figura 6.3.1: Accuracy Experimento 3.1

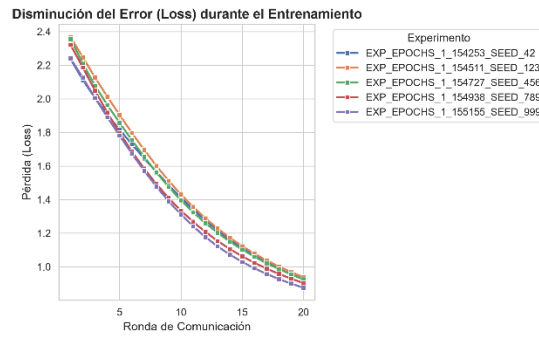


Figura 6.3.2: Loss Experimento 3.1

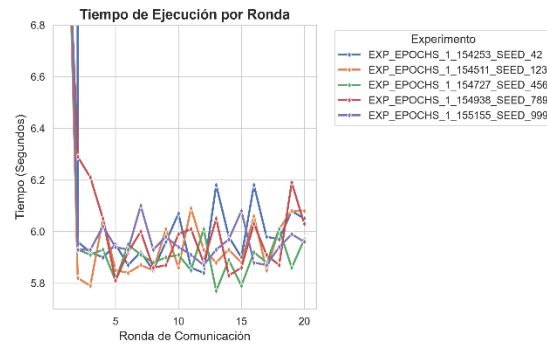


Figura 6.3.3: Tiempo Ejecución por Ronda Experimento 3.1

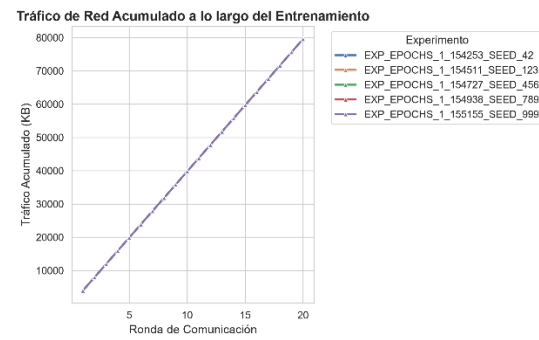


Figura 6.3.4: Tráfico Acumulado Experimento 3.1

6.3.2. Simbiosis y Asimilación del FedAvg Ponderado 1:10 y 1:50 (Exp. 3.2)

Caso A (Escenario 1:10):

Se utilizan 3 clientes Android con 100 muestras cada uno y 2 clientes simulados en Python con 1000 muestras por cliente.

En la Ronda 20 se alcanza una Accuracy de $91.56\% \pm 0.15\%$, lo que supone una mejora notable frente al 77.93% del escenario 1:1. Esto se explica porque, aunque cada terminal Android aporta solo el 4.3% del modelo global, los dos clientes Python concentran el 86.9% de los datos, estabilizando rápidamente el descenso de gradiente mediante la ponderación de McMahan.

El tráfico por ronda es de 3979.26 KB (3.98 MB), con un acumulado en 20 rondas de 79585.16 KB (79.59 MB) confirmando que el tráfico depende solo del número de clientes y no del volumen de datos procesado.

El tiempo medio de entrenamiento local es de $0.48 \text{ s} \pm 0.05 \text{ s}$, ligeramente superior al de la fase anterior por el mayor volumen de imágenes procesadas en el PC, y el tiempo total por ronda se estabiliza en $5.94 \text{ s} \pm 0.35 \text{ s}$.

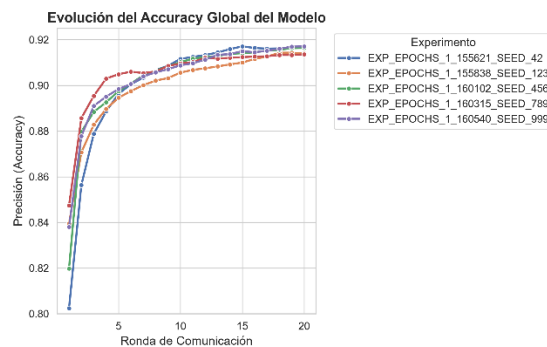


Figura 6.3.5: Accuracy 1:10 Experimento 3.2

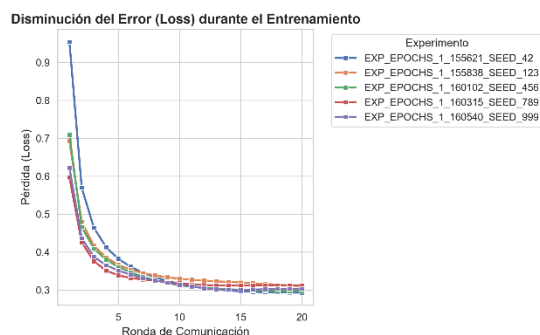


Figura 6.3.6: Loss 1:10 Experimento 3.2

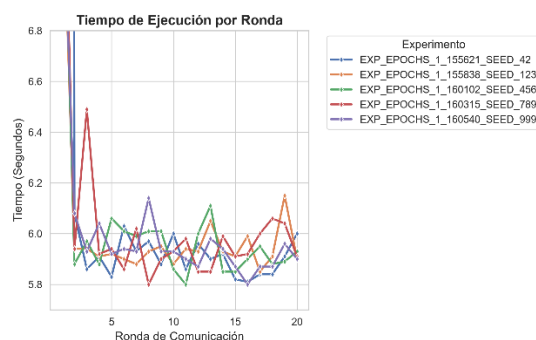


Figura 6.3.7: Tiempo Ejecución por Ronda 1:10 Experimento 3.2

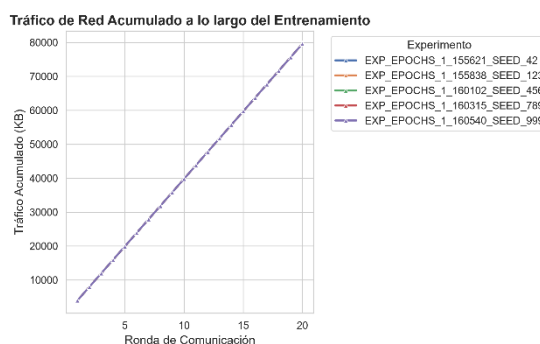


Figura 6.3.8: Tráfico Acumulado 1:10 Experimento 3.2

Caso B (Escenario 1:50):

En esta última casuística de experimentación tendremos los 3 mismos clientes Android con sus 100 muestras y los 2 clientes simulados en Python pero con 5000 muestras cada uno.

La Accuracy final en la Ronda 20 es $95.63\% \pm 0.20\%$ (pico máximo 95.84% en la semilla 789), prácticamente igual al techo asintótico de un modelo entrenado

en servidores puros (95.61% en Exp 2.4 con N=2). Esto confirma el poder de la ponderación por volumen de FedAvg: los clientes Python aportan un 97.08% del peso en cada agregación (97.08%), dando a los móviles Android una generalización excelente (95.63%) sin sobrecargar su hardware.

El tráfico por ronda es de 3.98 MB, con un acumulado total de 7959 MB.

El tiempo de entrenamiento local es $0.69 \text{ s} \pm 0.12 \text{ s}$, ligeramente superior a los 0.48 s de la fase con 1.000 muestras, reflejando el mayor coste de procesar 5.000 imágenes por bloque.

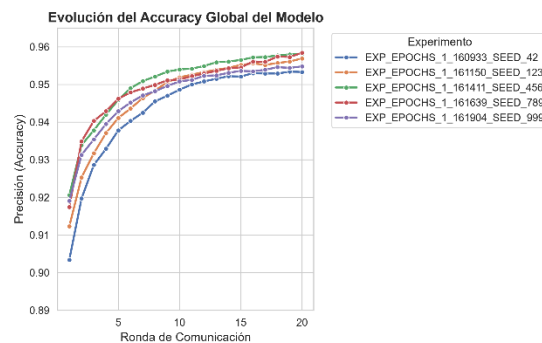


Figura 6.3.9: Accuracy 1:50 Experimento 3.2

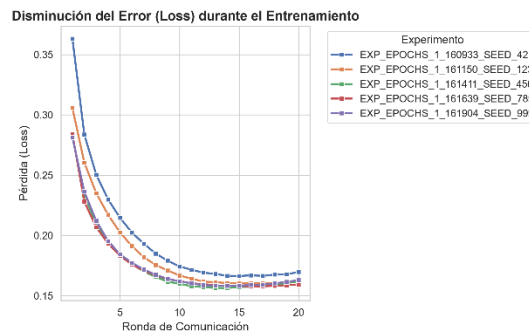


Figura 6.3.10: Loss 1:50 Experimento 3.2

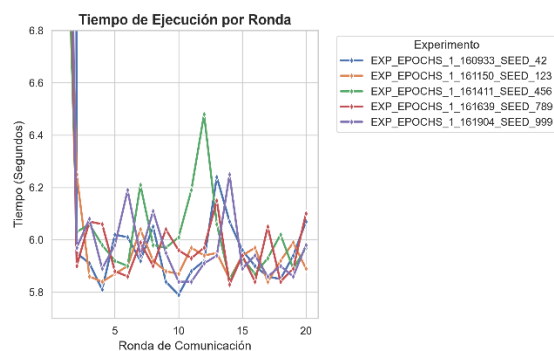


Figura 6.3.11: Tiempo Ejecución por Ronda 1:50 Experimento 3.2



Figura 6.3.12: Tráfico Acumulado 1:50 Experimento 3.2

Escenario Desequilibrio (Android:Python)	Muestras Totales	Peso Relativo (Android / Python)	Accuracy Final	Tiempo medio de Entrenamiento	Tráfico por Ronda (KB)
1:1	500	20/20%	77.93%	0.42s	3979.26
1:10	2300	4.3/43.5%	91.56%	0.48s	3979.26
1:50	10300	0.97/48.54	95.63%	0.69s	3979.26

Tabla 3: Métricas Experimentos 3.1 y 3.2

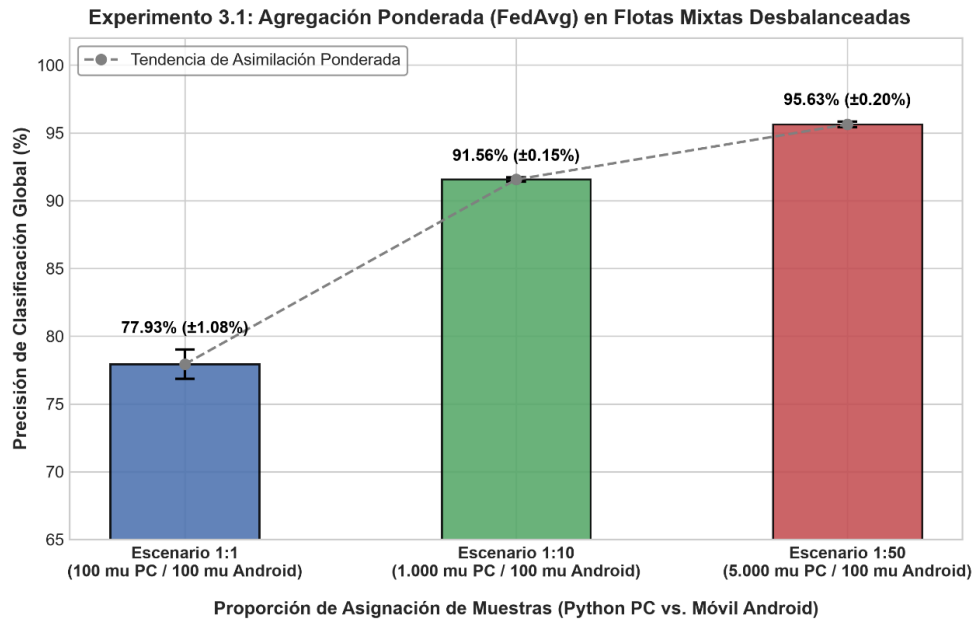


Figura 6.3.13: Gráfica Proporción Asignación Muestras Experimentos 3.1 y 3.2

La conclusión que se saca solo del peso relativo de cada cliente es la siguiente:

- Escenario 1:1: Todos los dispositivos mandan por igual. Un teléfono Android vale exactamente lo mismo para el servidor que un ordenador Python.
- Escenario 1:10: Aquí el servidor ya "escucha" mucho más a los PC. El modelo de un solo ordenador Python vale lo mismo que el de 10 teléfonos Android juntos.
- Escenario 1:50: Los dos ordenadores Python dominan en conjunto el 97.08% de la decisión del servidor ($48.54\% \times 2$). Cada teléfono Android aporta menos de un 1% del modelo.

El análisis del Peso Relativo en el Escenario 1:50 demuestra el fenómeno de simbiosis o asimilación positiva en FedAvg: aunque los dispositivos móviles físicos Android representan una fracción prácticamente marginal del consenso global (0.97% de peso por teléfono), no generan inestabilidad algorítmica ni divergencia. Al contrario, se benefician gratuitamente del 97% de conocimiento transferido por los nodos de alta capacidad (Python), alcanzando un 95.63% de precisión de clasificación sin haber tenido que gastar su batería ni su CPU entrenando más de 100 imágenes locales.

Parte IV

Conclusiones y Líneas Futuras

Capítulo 7

Conclusiones y Trabajo Futuro

Contenido

7.1.	Conclusiones del Proyecto	86
7.2.	Cumplimiento de Objetivos.....	87
7.3.	Líneas Futuras.....	88
7.3.1.	Estrategias Asíncronas de Agregación (Mitigación del <i>Straggler Effect</i>)	88
7.3.2.	Sustitución de HTTP REST por gRPC y Protocol Buffers (ProtoBuf)	89
7.3.3.	Participación Parcial Dinámica y Tolerancia al Dropout.....	89
7.3.4.	Aprendizaje Federado Personalizado (Personalized Federated Learning, PFL)	90

7.1. Conclusiones del Proyecto

A partir de los experimentos y mediciones realizadas sobre la plataforma de Aprendizaje Federado, se han extraído las siguientes conclusiones fundamentales:

- El ancho de banda y la sincronización como verdaderos cuellos de botella: Históricamente, el entrenamiento de modelos de *Machine Learning* ha estado limitado por la capacidad de cómputo. Sin embargo, este proyecto demuestra que en entornos federados el coste de CPU pasa a un segundo plano. El verdadero factor limitante es la capa de red. El intercambio constante de los pesos del modelo entre el servidor y los clientes satura el ancho de banda, y

la necesidad de sincronizar a todos los nodos en cada ronda genera tiempos de inactividad significativos.

- Eficiencia superior de la cuantización combinada con compresión: Para mitigar el problema de las comunicaciones, se ha demostrado que la combinación de técnicas de cuantización a 16 bits (F16) y algoritmos de compresión estándar (como ZIP) resulta altamente efectiva. La cuantización reduce a la mitad la huella de memoria de los tensores sin una degradación perceptible en el rendimiento predictivo del modelo. Al añadir la compresión ZIP sobre las cargas útiles, se logra una reducción drástica del tráfico de red, acelerando sustancialmente las rondas de comunicación.
- Viabilidad de dispositivos móviles en flotas híbridas desbalanceadas: Se ha comprobado que los dispositivos de borde (*Edge*), como los teléfonos móviles, poseen la capacidad de cómputo suficiente para ejecutar entrenamientos locales de manera eficiente. La arquitectura ha demostrado ser robusta frente a la heterogeneidad del hardware y al desbalanceo de datos inherente a una flota híbrida, confirmando que el Aprendizaje Federado es una solución técnica viable para aprovechar el potencial de dispositivos distribuidos sin comprometer la privacidad.

7.2. Cumplimiento de Objetivos

El desarrollo de este proyecto ha permitido satisfacer con éxito los objetivos planteados al inicio del mismo:

- Despliegue y validación arquitectónica: Se ha diseñado, implementado y puesto en producción una arquitectura funcional de Aprendizaje Federado capaz de orquestar múltiples nodos heterogéneos.
- Identificación de métricas y cuellos de botella: Se han cumplido los objetivos analíticos al perfilar el consumo de recursos (CPU, red, memoria), identificando claramente las limitaciones de comunicación frente a las de cómputo.

- Implementación de optimizaciones: La integración de mecanismos de cuantización (F16) y compresión (ZIP) ha dado respuesta al objetivo de optimizar el tráfico de red, mejorando la eficiencia general del sistema.

En definitiva, la plataforma resultante no solo prueba la viabilidad del Aprendizaje Federado en escenarios reales, sino que sienta una base sólida para futuras optimizaciones orientadas a entornos de producción masivos.

7.3. Líneas Futuras

A pesar de los sólidos resultados obtenidos en términos de eficiencia de red y validación de la arquitectura híbrida, el paradigma del Aprendizaje Federado se encuentra en constante evolución. Para escalar la solución desarrollada hacia un entorno de producción masivo y de nivel industrial, se proponen las siguientes líneas de investigación y optimización telemática.

7.3.1. Estrategias Asíncronas de Agregación (Mitigación del *Straggler Effect*)

El diseño actual fundamenta su orquestación en el protocolo síncrono FedAvg, el cual obliga al servidor central a mantener un estado de bloqueo inactivo (idle) hasta recepcionar la actualización del dispositivo más lento de la cohorte. Como se demostró empíricamente en el Bloque 1, este efecto rezagado (*Straggler Effect*) malgasta hasta el 78% del tiempo total del ciclo.

Una línea de continuación natural consiste en la implementación de protocolos de agregación asíncrona, tales como FedAsync [24]. En esta topología, el servidor no espera a reunir un número K de participantes, sino que actualiza el modelo maestro de forma continua cada vez que recibe una transmisión individual. Para evitar que las actualizaciones de clientes extremadamente lentos corrompan el modelo (debido al desfase de rondas), se requeriría el diseño de una función de penalización por obsolescencia (*staleness function*), la cual ponderaría a la baja los gradientes desfasados temporalmente.

7.3.2. Sustitución de HTTP REST por gRPC y Protocol Buffers (ProtoBuf)

Si bien la interfaz HTTP REST implementada mediante FastAPI ha demostrado ser altamente interoperable y robusta, el estándar HTTP/1.1 y el empaquetado multiparte presentan ineficiencias en escenarios de latencia crítica.

Se plantea como mejora arquitectónica la migración del canal de comunicaciones hacia el protocolo gRPC operando sobre HTTP/2, combinado con la serialización binaria estricta de Protocol Buffers (ProtoBuf) [25]. Esta actualización permitiría eliminar por completo el mecanismo de sondeo asíncrono (*Polling*) que actualmente ejecutan los clientes, sustituyéndolo por flujos bidireccionales (*Bidirectional Streaming*). El servidor podría empujar (*push*) el modelo global hacia los terminales instantáneamente en el momento exacto en que comienza la ronda, reduciendo el overhead de cabeceras HTTP y unificando el esquema de serialización de tensores entre Python y Android.

7.3.3. Participación Parcial Dinámica y Tolerancia al Dropout

En despliegues comerciales compuestos por millones de terminales (como redes de smartphones o sensores IoT), resulta técnica y energéticamente inviable exigir que la totalidad de la flota participe en cada iteración. Se propone el diseño de un algoritmo de selección dinámica de clientes. El servidor orquestador seleccionaría aleatoriamente a un subconjunto reducido de dispositivos activos (fracción de participación $C \in [0.1, 0.5]$) basándose en métricas de disponibilidad física: nivel de batería restante, conexión exclusiva a redes Wi-Fi (evitando el consumo de datos celulares del usuario) y estado de inactividad de la pantalla (*idle state*). Asimismo, esto obligaría a perfeccionar los mecanismos de tolerancia a fallos para manejar desconexiones abruptas (*Dropout*) sin necesidad de descartar la ronda completa.

7.3.4. Aprendizaje Federado Personalizado (Personalized Federated Learning, PFL)

Los resultados del Bloque 2 evidenciaron que, bajo distribuciones de datos patológicas y fuertemente sesgadas (Non-IID), el algoritmo FedAvg clásico sufre divergencia de pesos, oscilaciones en la curva de aprendizaje y una notable degradación de la precisión final.

Para resolver este obstáculo estadístico, una vía de investigación de gran impacto consistiría en la evolución hacia el Aprendizaje Federado Personalizado (PFL) [26]. En lugar de forzar a todos los clientes a adoptar un modelo global idéntico, la arquitectura PFL propone un enfoque de meta-aprendizaje o ajuste fino (*fine-tuning*). El sistema compartiría las capas base de extracción de características neuronales con el servidor central, pero mantendría las últimas capas de clasificación bloqueadas y exclusivas en cada dispositivo local. De este modo, la red neuronal final se adaptaría de forma especializada al comportamiento idiosincrásico de cada usuario individual, mejorando la tasa de acierto y eliminando las interferencias causadas por la heterogeneidad estadística de la red.

Parte V

Apéndices y Bibliografía

Apéndice A.

Configuración y Despliegue del entorno

CREACIÓN DE ENTORNO VIRTUAL PARA PYTHON

Se crea un entorno virtual para crear espacios aislados (directorios independientes) que permiten gestionar las dependencias y bibliotecas de cada proyecto por separado. Además, evita conflictos entre librerías, asegura la reproducibilidad del código y protege la instalación global de Python del sistema.

1. Crear el entorno virtual.

En terminal comando: ***python -m venv .venv***

2. Seleccionar el intérprete, en Visual Studio Code:

Ctrl + Shift + P

Buscar Python: ***Select Interpreter***

Seleccionar opción que apunta a carpeta ***.venv*** en tu directorio de trabajo.

3. Activar el entorno:

.venv\Scripts\activate

INSTALACIÓN DE TENSORFLOW

Tensorflow es una biblioteca de código abierto desarrollada por Google para aprendizaje automático (machine learning) e inteligencia artificial, diseñada para facilitar la creación, entrenamiento y despliegue de modelos de redes neuronales.

1. Asegurar que en el terminal estemos con el entorno virtual activo.

2. Actualizar pip:

python -m pip install --upgrade pip

3. Instalar TensorFlow:

```
pip install tensorflow
```

4. Verifica la instalación:

```
python -c "import tensorflow as tf; print(tf.__version__)"
```

INSTALACIONES PARA SERVIDOR PYTHON

Instalar FastAPI y su servidor web (uvicorn), además de una librería para manejar la subida de archivos.

Esto permite crear, desarrollar y ejecutar APIs web modernas, asíncronas y de alto rendimiento en Python.

FastAPI facilita la creación rápida de servicios con validación de datos automática, mientras que Uvicorn actúa como el servidor ASGI (Asynchronous Server Gateway Interface) necesario para servir la aplicación y verla en tiempo real.

```
pip install fastapi uvicorn python-multipart numpy
```

PREPARAR PETICIONES API REST

Para que Python pueda hacer peticiones a tu servidor, necesitamos instalar la librería estándar de redes en Python.

```
pip install requests
```

INSTALACIÓN MATPLOTLIB

Para que se puedan crear gráficas automatizadas recogiendo los resultados guardados en los archivos .csv para la experimentación necesitamos instalar la siguiente biblioteca.

```
pip install pandas matplotlib seaborn
```

CREACIÓN Y CONFIGURACIÓN DE PROYECTO EN ANDROID STUDIO

Paso 1: Los Permisos y la Seguridad (AndroidManifest.xml)

En la columna de la izquierda de Android Studio, ve a la carpeta **app** -> **manifests** y abre el archivo **AndroidManifest.xml**.

Tenemos que añadir dos cosas:

1. El permiso de Internet.
2. Un permiso especial llamado **usesCleartextTraffic="true"**. Esto es necesario porque desde Android 9, Google bloquea por seguridad cualquier tráfico que no sea HTTPS (cifrado). Como nuestro servidor local usa HTTP normal, tenemos que decirle a Android que confíe en él.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.INTERNET" />

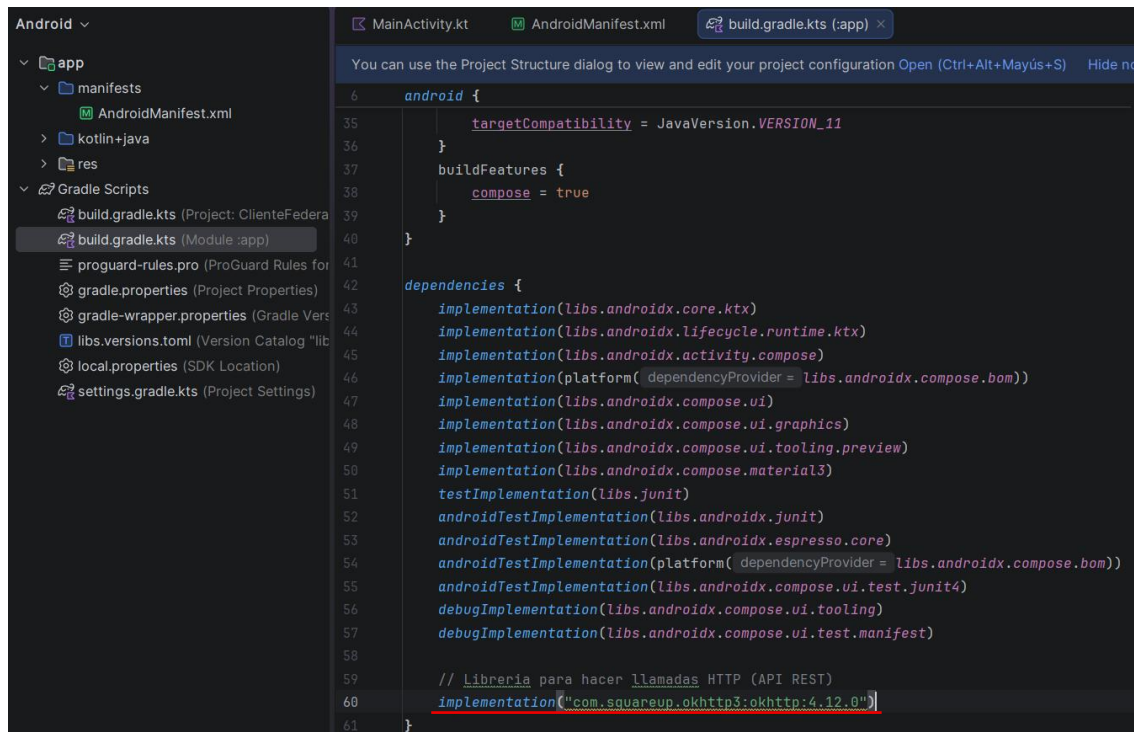
    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="ClienteFederado"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.ClienteFederado"
        tools:targetApi="31"
        android:usesCleartextTraffic="true">

        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="ClienteFederado"
            android:theme="@style/Theme.ClienteFederado">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
```

Paso 2: La Librería de Red (build.gradle.kts)

Para hacer peticiones HTTP en Android de forma sencilla, la industria utiliza una librería llamada **OkHttp**. Tenemos que descargarla.



CONFIGURAR TENSORFLOW LITE EN ANDROID STUDIO

TensorFlow Lite es la librería oficial, ligera y nativa de Google para aplicaciones móviles.

Por tanto, tendremos que modificar su archivo Gradle. **build.gradle.kts (Module:app)**

Se añade la librería de TFLite.

```
// Librería TensorFlow Lite para Android
implementation("org.tensorflow:tensorflow-lite:2.16.1")
implementation("org.tensorflow:tensorflow-lite-select-tf-ops:2.16.1") // soportar operaciones de entrenamiento (Flex)
```

Además, se añade una regla para que no comprima nuestro archivo *.tflite* (TFLite necesita leerlo en crudo para ser rápido).

```
// No comprimir archivos .tflite al empaquetar la app
androidResources {
    |   noCompress += "tflite"
    }
}
```

Bibliografía

- [1] Martin Abadi et al., «TensorFlow: Large-Scale Machine Learning for Heterogeneous Distributed Systems [Documentación de software].», 2015.
- [2] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, «Edge computing: Vision and challenges.,» *IEEE Internet of Things Journal*, vol. 3, no. 5 pp. 637-646, Oct. 2016.
- [3] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, «Communication-Efficient Learning of Deep Networks from Decentralized Data,» *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 1273-1282, 2017.
- [4] Seongeun Han, «A Robust Aggregation Approach for Heterogeneous Federated Learning,» *Fourteenth International Conference on Ubiquitous and Future Networks (ICUFN)*, pp. 493-497, 2023.
- [5] S. Ermut, «Federated Learning: 7 casos de uso y ejemplos,» *AIMultiple*, [Online] <https://aimultiple.com/federated-learning>, Jun. 11, 2026. [Consultada: 13/08/2026].
- [6] A. Géron, *Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems* (2.^a ed.), Sebastopol, California: O'Reilly Media, 2019.
- [7] D. P. Kingma and J. Ba, «Adam: A method for stochastic optimization.,» *Proceedings of the 3rd International Conference on Learning Representations (ICLR).*, 2015.
- [8] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, «Federated Learning: Challenges, Methods, and Future Directions,» *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50-60, 2020.
- [9] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, «Federated Learning with Non-IID Data,» 2018, *arXiv:1806.00582*.

- [10] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, «QSGD: Communication-Efficient SGD via Gradient Quantization and Encoding,» in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 1709-1720, 2017.
- [11] S. Ramírez, «FastAPI,» 2018. [Online]. <https://fastapi.tiangolo.com/>. [Consultada 13/08/2026].
- [12] "NumPy Reference Manual (Version 2.5)," *NumPy*, 2026. [Online]. <https://numpy.org/>. [Consultada: 14/08/2026].
- [13] «LiteRT: High-Performance On-Device Machine Learning Framework,» *Google AI Edge*, 2026. [Online] <https://developers.google.com/edge/litert-lm>. [Consultada: 14/08/2026].
- [14] R. T. Fielding, «Architectural styles and the design of network-based software architectures,» *Ph.D. dissertation, Department of Information and Computer Science, University of California, Irvine, Irvine, CA, USA, 2000, pp. 116-134..*
- [15] J. Tan, Y. Chen, Z. Liu, B. Ren, S. L. Song, X. Shen, and X. Liu, «Toward Efficient Interactions between Python and Native Libraries,» in *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2021)*, 2021, pp. 135–147.
- [16] J. L. Hennessy and D. A. Patterson, *Arquitectura de Computadores: Un Enfoque Cuantitativo*, 6a ed., Ciudad de México: McGraw-Hill, 2019, pp. 42-45.
- [17] F. Liang, Z. Zhang, H. Lu, V. C. M. Leung, Y. Guo, and X. Hu, «Communication-Efficient Large-Scale Distributed Deep Learning: A Comprehensive Survey,» 2024. *arXiv:2404.06114* [Online]. <https://arxiv.org/abs/2404.06114>. [Consultada: 14/08/2026].

- [18] «Asynchronous Server Gateway Interface (ASGI) Specification (Version 3.0),» *Django Software Foundation*, 2026. [Online]. <https://about.readthedocs.io>. [Consultada: 15/08/2026].
- [19] Y. Deng, S. Zhou, H. F. Wang, Y. J. Dong, S. & Z. A, «A Survey of Deep Learning on Mobile Devices: Applications, Optimizations, Challenges, and Research Opportunities.,» *IEEE*, 110(3), vol. 110, no. 3, pp. 334-361, 2022.
- [20] Martín Abadi et al., «TensorFlow: A system for large-scale machine learning,» *arXiv:1605.08695*, 2016.
- [21] V. P. Cid, «Buenas prácticas en visualización de datos,» *Universitat Oberta de Catalunya (UOC), Barcelona, España*, 2017.
- [22] J. D. Hunter, "*Matplotlib: A 2D Graphics Environment*", Computing in Science & Engineering, no. 3 ed., vol. vol. 9, 2007, pp. 90-95.
- [23] David Hill et al., "*Machine learning and reproducibility impact of random numbers.*", 38th European Simulation and Modelling Conference (ESM) ed., San Sebastian: HAL Open Science, 2024, pp. 65-70.
- [24] C. Xie, S. Koppel, and I. Goodfellow, «Asynchronous Federated Optimization,» *arXiv:1903.03934*, pp. 1-6, 2020.
- [25] M. Niswar, R. Ahmad, S. Anshari, B. Idrus, and A. Achmad, «Performance evaluation of microservices communication with REST, GraphQL, and gRPC,» *International Journal of Electronics and Telecommunications*, vol. 70, no. 2, pp. 261-267, 2024.
- [26] A. Tan, Z. Zhang, H. Yu, C. Liang, and Y. Qin, «Towards personalized federated learning: A taxonomy and review. » *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 6, pp. 5784-5801, Jun. 2023.